완 료 보 고 서

RISC-V RV32I Single Cycle CPU 설계

작 성 자 | 최기훈

프로젝트 기간 | 2026.05.19 (월) ~ 2026.05.27 (화)

발 표 일 | 2026.05.26 (월)

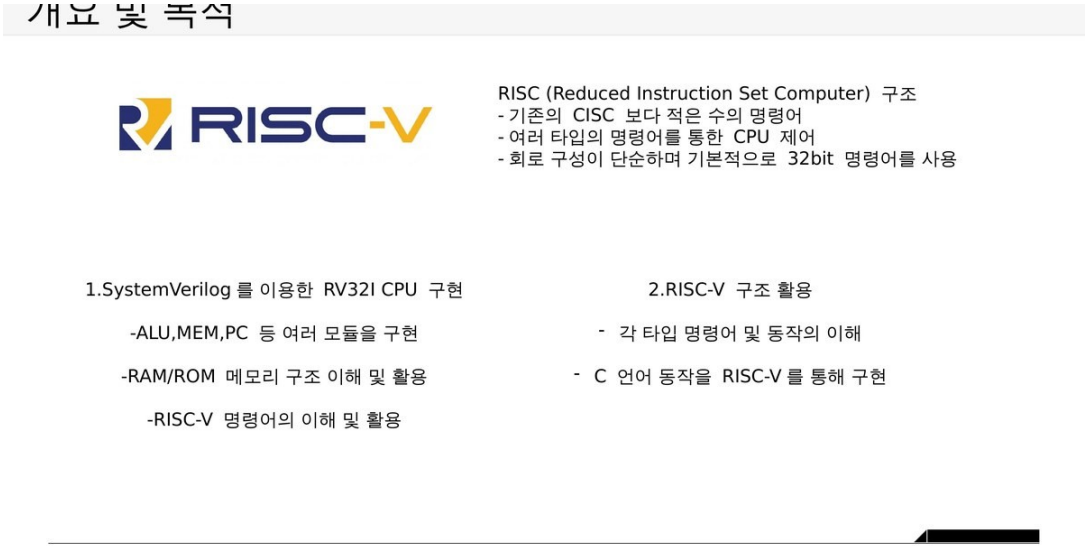
사 용 언 어 | SystemVerilog (IEEE 1800-2017)

1. 개요 (Overview)

1.1 프로젝트 배경 및 목적

RISC(Reduced Instruction Set Computer) 아키텍처는 간단하고 균일한 명령어 셋으로 하드웨어 구조를 단순화하고 컴파일러 최적화를 용이하게 한다는 철학에서 출발한다. RISC-V 는 이 철학을 계승한 오픈소스 ISA(Instruction Set Architecture)로, 특히 RV32I 베이스 정수 명령어 셋은 32 비트 고정 길이 명령어 36 종을 정의하며, 학습 및 연구 목적의 CPU 설계에 널리 활용된다.

본 프로젝트는 SystemVerilog 를 사용하여 RISC-V RV32I 명령어 셋을 완전히 지원하는 Single Cycle CPU 를 설계하고, 기능 시뮬레이션으로 검증하는 것을 목표로 한다. Single Cycle 방식은 매 클럭 사이클마다 하나의 명령어를 완전히 처리하는 구조로, 파이프라인의 복잡도 없이 전체 데이터 흐름을 명확하게 이해하기에 적합하다.



[그림] 프로젝트 개요 및 목표 — RISC-V 구조 활용 목적

1.2 설계 목표 및 요구사항

- SystemVerilog 를 이용한 RV32I CPU 전체 모듈 구현 — ALU, Register File, PC, 메모리, Control Unit
- R / I / S / L(Load) / B / UL(LUI) / UA(AUIPC) / J(JAL) / JL(JALR) 총 9 가지 명령어 타입 완전 지원
- C 언어 for 루프 동작을 RISC-V 어셈블리로 변환하여 1~10 합산(1to10 adder) 프로그램 실행 검증
- Testbench 를 통한 기능 시뮬레이션 — 10ns(50MHz) 클럭, 2 사이클 리셋, 20 사이클 이상 자유 실행
- 명령어 메모리는 $readmemh 로 외부 .mem 파일을 로드하는 구조로 유연한 테스트 지원

1.3 개발 일정 요약

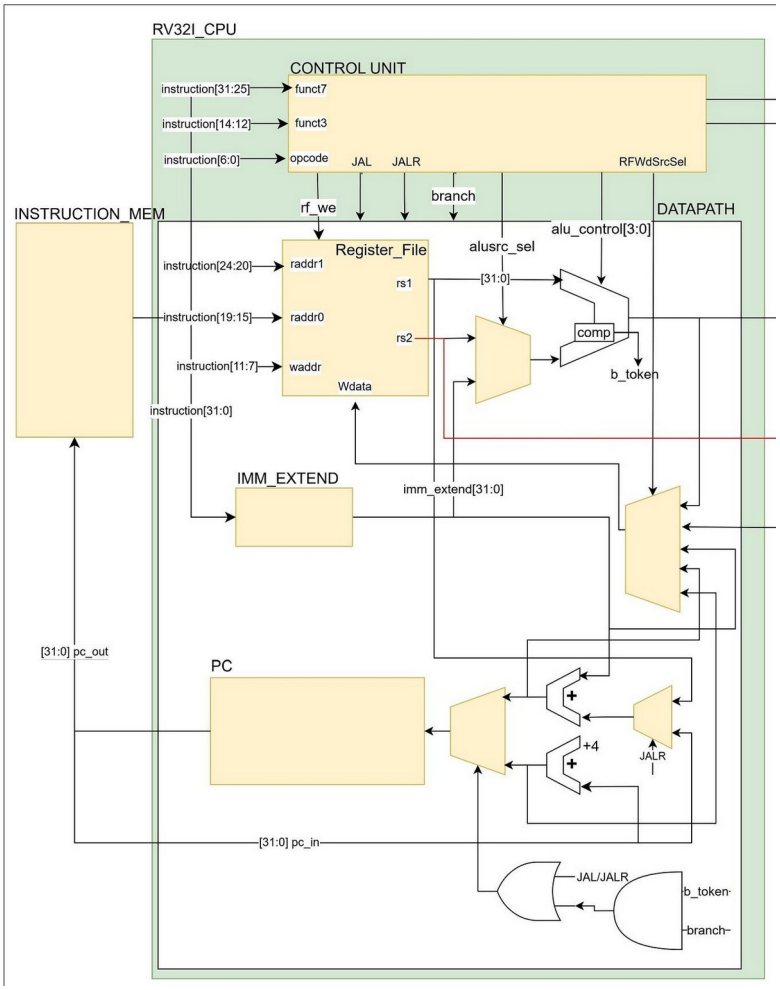
날짜	단계	주요 내용
05/19 (월)	기획·설계	ISA 분석, SoC 아키텍처 확정, define.vh 작성
05/20 (화)	RTL 설계	register_file, ALU 구현 (R-Type 10 종)
05/21 (수)	RTL 설계	imm_extend, mux_wb, program_counter 구현
05/22 (목)	RTL 설계	control_unit, rv32i_cpu, top_rv32i_soc, instruction_mem 완성
05/23 (금)	시뮬레이션	R / I / S / L 타입 명령어 파형 검증
05/24 (토)	시뮬레이션	B / U / J / JL 타입 검증, 1to10 adder 구현

05/25 (일)	트러블슈팅	b_taken X 값 버그 수정, 전체 통합 시뮬레이션
05/26 (월)	발표	RISC-V RV32I 설계 결과 발표
05/27 (화)	산출물 정리	발표 자료 보완, 보고서/일지/일정표 작성, 패키징

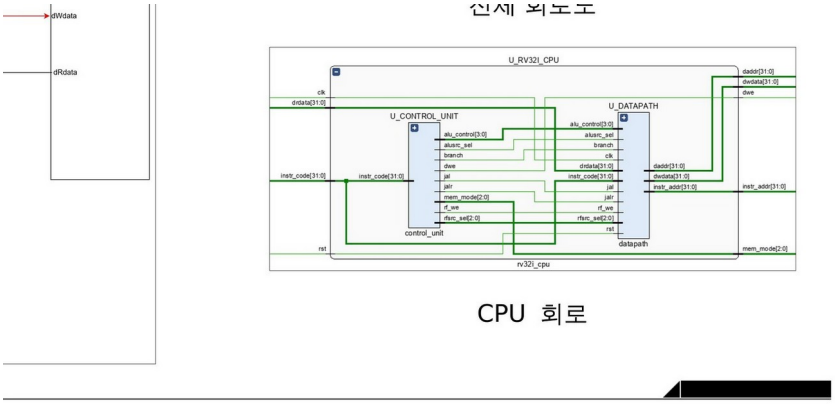
2. 아키텍처 설계 (Architecture Design)

2.1 전체 SoC 시스템 구조

전체 시스템은 top_rv32i_soc 를 최상위 모듈로, Instruction ROM (instruction_mem), CPU (rv32i_cpu), Data RAM (data_mem) 세 블록으로 구성된다. CPU 내부는 Control Unit 과 Datapath 로 분리되며, Control Unit 이 명령어를 디코딩하여 생성한 제어 신호가 Datapath 의 모든 MUX 와 레지스터 동작을 결정한다.



[그림] Design B/D — 전체 데이터패스 블록 다이어그램 (RV32I_CPU 내부 구조)



[그림] 전체 회로도 및 CPU 회로 (Vivado 시뮬레이터)

모듈명	역할 및 주요 신호

top_rv32i_soc	최상위 통합 모듈. clk, rst 입력만 노출. instruction_mem, rv32i_cpu, data_mem 세 모듈 연결
instruction_mem	32-entry ROM. $readmemh 로 .mem 파일 로드. instr_code = instr_rom[instr_addr[31:2]] (워드 단위 인덱싱)
rv32i_cpu	control_unit + datapath 통합. .* 와일드카드 포트 연결. 내부 신호: rf_we, branch, jal, jalr, alusrc_sel, alu_control[3:0], rfsrc_sel[2:0]
control_unit	순수 조합 논리. opcode[6:0], funct3[2:0], funct7[6:0] 입력 → 8 개 제어 신호 출력. always_comb + case(opcode) 구조
datapath	Register File, ALU, IMM_EXTEND, mux_wb(5to1), mux_2x1, program_counter 포함. daddr=alu_result, dwdata=rs2
data_mem	64-entry RAM. always_ff 동기 쓰기(SB/SH/SW), always_comb 비동기 읽기(LB/LH/LW/LBU/LHU). daddr[31:2] 워드 인덱싱
tb_rv32i	10ns 클럭(50MHz) 생성, negedge 기준 2 사이클 리셋 후 20 사이클 실행. $stop 으로 시뮬레이션 종료

2.2 Single Cycle 방식의 동작 원리

Single Cycle 구조에서는 매 클럭의 상승 엣지(posedge clk)마다 하나의 명령어가 완전히 처리된다. Instruction Fetch 부터 Write Back 까지의 5 단계가 모두 조합 논리로 처리되며, 오직 PC 와 Register File, Data Memory 의 쓰기만 순차 논리(always_ff)로 구현된다.

클럭 한 주기 내의 신호 흐름은 다음과 같다: ① PC 에서 명령어 주소 출력 → ② instruction_mem 에서 32 비트 명령어 로드 → ③ control_unit 에서 제어 신호 생성(조합) → ④ register_file 에서 rs1, rs2 비동기 읽기 → ⑤ imm_extend 에서 즉시값 생성 → ⑥ ALU 연산 수행 → ⑦ 필요 시 data_mem 접근 → ⑧ 결과를 WB MUX 로 선택하여 다음 posedge 에 register_file 쓰기 → ⑨ 다음 PC 값 결정.

[코드] top_rv32i_soc.sv — 최상위 SoC 모듈

```
`timescale 1ns / 1ps

module top_rv32i_soc (
    input clk,
    input rst
);
    logic [31:0] instr_code, instr_addr, daddr, dwdata, drdata;
    logic [2:0] mem_mode;
    logic dwe;
    instruction_mem U_INSTR_ROM (.*);

    rv32i_cpu U_RV32I_CPU (.*);

    data_mem U_data_RAM (.*);
endmodule
```

[코드] rv32i_cpu.sv — CPU 통합 모듈 (control_unit + datapath)

```
`timescale 1ns / 1ps

module rv32i_cpu (
    input  logic        clk,
    input  logic        rst,
    input  logic [31:0] instr_code,
    input  logic [31:0] drdata,
    output logic [31:0] instr_addr,
    output logic [ 2:0] mem_mode,
    output logic        dwe,
    output logic [31:0] daddr,
    output logic [31:0] dwdata
);
    logic rf_we, branch, alusrc_sel;
    logic [3:0] alu_control;
```

```
    logic [2:0] rfsrc_sel;
    logic jal, jalr;
    control_unit U_CONTROL_UNIT (.*);
    datapath U_DATAPATH (.*);
endmodule
```

3. 상세 설계 (Detailed Design)

3.1 define.vh — 매크로 정의

define.vh 는 프로젝트 전체에서 사용하는 opcode, ALU 연산 코드, 메모리 모드 등을 매크로로 정의한다. 매직 넘버 사용을 방지하고 코드 가독성을 높이며, 유지보수 시 한 곳에서 변경이 가능하도록 설계하였다.

[코드] define.vh — 전체 매크로 정의

```
// OP-CODE
`define R_TYPE 7'b011_0011
`define S_TYPE 7'b010_0011
`define IL_TYPE 7'b000_0011
`define I_TYPE 7'b001_0011
`define B_TYPE 7'b110_0011
`define UL_TYPE 7'b011_0111 // LUI
`define UA_TYPE 7'b001_0111 // AUIPC
`define J_TYPE 7'b110_1111  // JAL
`define JL_TYPE 7'b110_0111 // JALR

// R-type,I-type instruction
// {funct7,funct3} = 10bit
`define ADD 4'b0_000
`define SUB 4'b1_000
`define SLL 4'b0_001
`define SLT 4'b0_010
`define SLTU 4'b0_011
`define XOR 4'b0_100
`define SRL 4'b0_101
`define SRA 4'b1_101
`define OR 4'b0_110
`define AND 4'b0_111

// S-type instruction
`define SB 3'b000
`define SH 3'b001
`define SW 3'b010

//IL-type instruction
`define LB 3'b000
`define LH 3'b001
`define LW 3'b010
`define LBU 3'b100
`define LHU 3'b101

// B-type instruction
`define BEQ 3'b000
`define BNE 3'b001
`define BLT 3'b100
`define BGE 3'b101
`define BLTU 3'b110
`define BGEU 3'b111
```

3.2 Control Unit

control_unit 은 32 비트 명령어를 입력받아 데이터패스를 제어하는 8 개의 신호를 출력하는 순수 조합 논리 모듈이다. always_comb 블록 내에서 case(opcode)로 9 가지 타입을 구분하며, 블록 진입 시 모든 신호를 default 값으로 초기화하여 래치 생성을 방지한다. 디버그 가시성을 위해 typedef enum 으로 opcode_dbg 신호를 선언하여 파형 뷰어에서 심볼릭 이름으로 표시되도록 하였다.

타입	rf_we	alusrc_sel	wr_en	branch	JAL	JALR	rfsrc_sel	RFWdSrc
R	1	0 (rs2)	0	0	0	0	000	ALU result
I (산술)	1	1 (imm)	0	0	0	0	000	ALU result

I (Load)	1	1 (imm)	0	0	0	0	001	drdata
S	0	1 (imm)	1	0	0	0	—	—
B	0	0	0	1	0	0	—	—
UL (LUI)	1	—	0	0	0	0	010	imm_extend
UA (AUIPC)	1	—	0	0	0	0	011	PC+imm
J (JAL)	1	—	0	0	1	0	100	PC+4
JL (JALR)	1	1 (imm)	0	0	1	1	100	PC+4

[코드] control_unit.sv — R-TYPE / I-TYPE 제어 신호 발생 로직

```
// R-TYPE: rf_we=1, alusrc_sel=0(rs2), alu_control={funct7[5],funct3}
`R_TYPE: begin
    rf_we       = 1'b1;
    alusrc_sel  = 0;          // ALU 두 번째 입력 = rs2
    alu_control = {funct7[5], funct3};  // {SUB 구분비트, 연산종류}
    rfsrc_sel   = 0;          // WB 선택: ALU 결과
    dwe         = 0;
end

// I-TYPE (산술): rf_we=1, alusrc_sel=1(imm)
`I_TYPE: begin
    rf_we      = 1'b1;
    alusrc_sel = 1'b1;        // ALU 두 번째 입력 = imm_extend
    if (funct3 == 3'b101)     // SRLI vs SRAI 구분: funct7[5] 사용
        alu_control = {funct7[5], funct3};
    else
        alu_control = {1'b0, funct3};
    rfsrc_sel = 0;            // WB 선택: ALU 결과
    dwe       = 0;
end
```

[코드] control_unit.sv — B-TYPE / UL·UA-TYPE / J·JL-TYPE 제어 신호 발생 로직

```
// B-TYPE: branch=1, rf_we=0, wr_en=0
`B_TYPE: begin
    rf_we       = 1'b0;
    branch      = 1;          // PC 분기 활성화 신호
    alusrc_sel  = 0;          // ALU 입력 = rs2 (비교용)
    alu_control = {1'b0, funct3};  // BEQ/BNE/BLT/BGE/BLTU/BGEU
    dwe         = 0;
end

// UL-TYPE (LUI): imm[31:12]을 rd 상위에 배치
// UA-TYPE (AUIPC): PC + imm[31:12] → rd
`UL_TYPE, `UA_TYPE: begin
    rf_we = 1'b1;
    if (opcode == `UL_TYPE) rfsrc_sel = 3'b010;  // WB: imm (LUI)
    else                    rfsrc_sel = 3'b011;  // WB: PC+imm (AUIPC)
    dwe = 0;
end

// J-TYPE (JAL), JL-TYPE (JALR)
`J_TYPE, `JL_TYPE: begin
    rf_we  = 1'b1;
    jal    = 1;               // JAL/JALR 공통: PC 점프 활성화
    if (opcode == `J_TYPE) jalr = 0;
    else                   jalr = 1;  // JALR: rs1+imm 기반 점프
    rfsrc_sel = 3'b100;       // WB: PC+4 (복귀 주소)
    dwe = 0;
end
```

3.3 ALU (Arithmetic Logic Unit)

ALU 는 4 비트 alu_control 을 입력받아 10 가지 연산을 수행하는 순수 조합 논리 모듈이다. alu_control 은 {funct7[5], funct3}으로 구성되며, funct7[5]=1 일 때 SUB/SRA 로 전환된다. b_taken 출력 신호는 B-Type 분기 조건 판단에 사용된다.

[코드] rv32i_datapath.sv — ALU 모듈 (산술/논리 연산 및 b_taken)

```
module alu (
    input logic [3:0] alu_control,
    input logic [31:0] rs1,
    input logic [31:0] rs2,
    output logic [31:0] alu_result,
    output logic b_taken
);
    always_comb begin
        alu_result = 0;
        case (alu_control)
            `ADD:  alu_result = rs1 + rs2;
            `SUB:  alu_result = rs1 - rs2;
            `SLL:  alu_result = rs1 << rs2;
            `SLT:  alu_result = ($signed(rs1) < $signed(rs2)) ? 1 : 0;
            `SLTU: alu_result = (rs1 < rs2) ? 1 : 0;
            `XOR:  alu_result = rs1 ^ rs2;
            `SRL:  alu_result = rs1 >> rs2;
            `SRA:  alu_result = $signed(rs1) >>> rs2[4:0];
            `OR:   alu_result = rs1 | rs2;
            `AND:  alu_result = rs1 & rs2;
        endcase
    end

always_comb begin
    b_taken = 1'b0;
    case (alu_control[2:0])
        `BEQ: begin
            if (rs1 == rs2) b_taken = 1'b1;
            else b_taken = 1'b0;
        end
        `BNE: begin
            if (rs1 != rs2) b_taken = 1'b1;
            else b_taken = 1'b0;
        end
        `BLT: begin
            if ($signed(rs1) < $signed(rs2)) b_taken = 1'b1;
            else b_taken = 1'b0;
        end
        `BGE: begin
            if ($signed(rs1) >= $signed(rs2)) b_taken = 1'b1;
            else b_taken = 1'b0;
        end
        `BLTU: begin
            if (rs1 < rs2) b_taken = 1'b1;
            else b_taken = 1'b0;
        end
        `BGEU: begin
            if (rs1 >= rs2) b_taken = 1'b1;
            else b_taken = 1'b0;
        end
    endcase
end
```

3.4 Register File

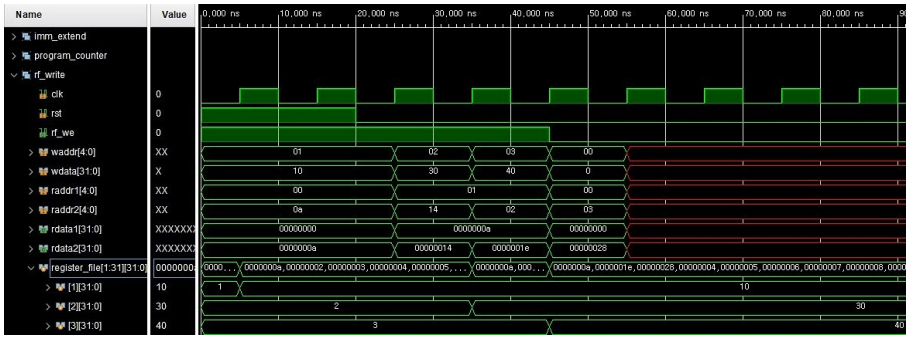
register_file 은 x0~x31 총 32 개의 32 비트 레지스터를 구현한다. x0 는 RISC-V 스펙에 따라 항상 0 을 반환하며 (하드와이어링), 쓰기는 posedge clk 에 동기식으로 수행된다. 읽기는 비동기 assign 으로 구현하여 같은 사이클 내에서 읽기와 쓰기가 동시에 가능하다.

[코드] rv32i_datapath.sv — register_file 모듈

```
module register_file (
    input  logic        clk,
    input  logic [ 4:0] raddr1,  // rs1
    input  logic [ 4:0] raddr2,  // rs2
    input  logic        rf_we,   // write enable
    input  logic [ 4:0] waddr,   // rd
```

```
    input  logic [31:0] wdata,   // rd write data
    output logic [31:0] rdata1,  // rs1 read data
    output logic [31:0] rdata2   // rs2 read data
);
    logic [31:0] register_file[1:31];

    always_ff @(posedge clk) begin
        if (rf_we) begin
            register_file[waddr] <= wdata;
        end
    end
    // x0는 항상 0 반환 (하드와이어링)
    assign rdata1 = (raddr1 != 0) ? register_file[raddr1] : 32'h0000_0000;
    assign rdata2 = (raddr2 != 0) ? register_file[raddr2] : 32'h0000_0000;
endmodule
```



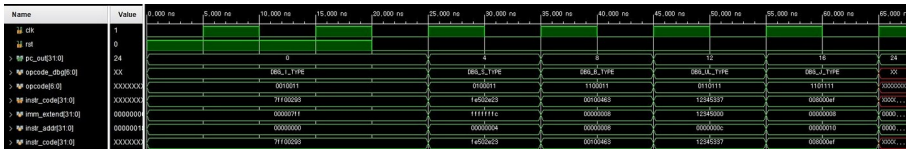
[그림] register_file 검증 파형 — posedge clk 에 rf_we=1 일 때 waddr/wdata 기록, rdata1/rdata2 비동기 읽기, x1→10, x2→30, x3→40 순차 쓰기 확인

3.5 IMM_EXTEND (즉시값 확장기)

imm_extend는 명령어 타입별로 흩어져 인코딩된 즉시값 비트를 재조합하고 32비트로 부호확장하는 모듈이다. B-Type과 J-Type의 즉시값은 비트가 분산되어 있어 재조합이 필요하며, 이는 하드웨어 구현 시 배선을 단순화하기 위한 RISC-V 스펙의 의도적 설계이다.

[코드] rv32i_datapath.sv — imm_extend 모듈 (6가지 타입별 즉시값 재조합)

```
module imm_extend (
    input  logic [31:0] instr_code,
    output logic [31:0] imm_extend
);
    always_comb begin
        imm_extend = 32'd0;
        case (instr_code[6:0])
            `S_TYPE:
                imm_extend = {{20{instr_code[31]}}, instr_code[31:25], instr_code[11:7]};
            `IL_TYPE, `I_TYPE, `JL_TYPE:
                imm_extend = {{20{instr_code[31]}}, instr_code[31:20]};
            `B_TYPE:
                imm_extend = {{20{instr_code[31]}},
                              instr_code[7],      // imm[11]
                              instr_code[30:25],  // imm[10:5]
                              instr_code[11:8],   // imm[4:1]
                              1'b0};              // imm[0] = 0 (2-byte align)
            `UL_TYPE, `UA_TYPE:
                imm_extend = {instr_code[31:12], 12'h000};
            `J_TYPE:
                imm_extend = {{12{instr_code[31]}},
                              instr_code[19:12],  // imm[19:12]
                              instr_code[20],     // imm[11]
                              instr_code[30:21],  // imm[10:1]
                              1'b0};              // imm[0] = 0
        endcase
    end
endmodule
```



[그림] imm_extend 검증 파형 — I(0x7FF) → S(0xFFFFFFFC) → B(0x8) → U(0x12345000) → J(0x8) 순서대로 타입별 즉시값 출력 확인

3.6 Program Counter (PC)

program_counter 는 다음 실행할 명령어 주소를 관리하는 모듈로, 두 개의 2-to-1 MUX 와 레지스터로 구성된다. 첫 번째 MUX(JALR MUX)는 jalr 신호에 따라 기준 주소를 현재 PC(JAL/B) 또는 rs1(JALR) 중에서 선택한다. 두 번째 MUX(SRC MUX)는 점프/분기 조건에 따라 pc_4 또는 pc_imm 을 pc_next 로 선택한다.

[코드] rv32i_datapath.sv — program_counter 모듈

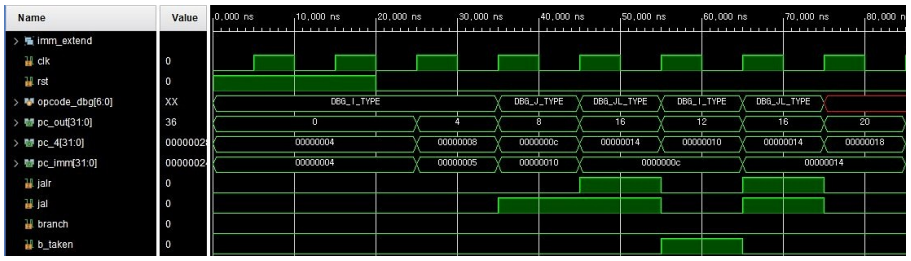
```
module program_counter (
    input         clk, rst, b_taken, branch, jal, jalr,
    input  [31:0] pc_in, rs1, imm_extend,
    output [31:0] pc_out, pc_imm, pc_4
);
    logic [31:0] pc_reg, pc_next, pc_jalr;

    assign pc_out = pc_reg;
    assign pc_imm = imm_extend + pc_jalr;  // JALR: rs1+imm, JAL/B: PC+imm
    assign pc_4   = pc_in + 32'd4;

    // JALR 선택 MUX: jalr=1 이면 rs1, 아니면 현재 PC
    mux_2x1 U_PC_JALR_MUX (.in0(pc_in), .in1(rs1), .sel(jalr), .out_mux(pc_jalr));

    // PC SRC MUX: jal/jalr/(branch&b_taken)이면 pc_imm, 아니면 pc_4
    mux_2x1 U_PC_SRC_MUX (
        .in0(pc_4), .in1(pc_imm),
        .sel((jal | jalr) | (branch & b_taken)),
        .out_mux(pc_next)
    );

    always_ff @(posedge clk, posedge rst) begin
        if (rst) pc_reg <= 0;
        else     pc_reg <= pc_next;
    end
endmodule
```



[그림] program_counter 검증 파형 — rst→PC=0 초기화, PC+4 순차 증가, JAL(jal=1) 점프, JALR(jalr=1) 레지스터 기반 복귀 동작

3.7 Data Memory

data_mem 은 64-entry 32 비트 RAM 으로, 주소는 daddr[31:2]를 워드 인덱스로 사용한다. 쓰기는 SB/SH/SW 세 가지 모드를 지원하며, daddr[1:0]를 이용해 바이트/하프워드 정렬 주소를 처리한다. 읽기는 always_comb 비동기로 구현되어 같은 사이클에 쓰기와 읽기가 발생해도 쓰기 전 데이터를 읽는 read-before-write 방식으로 동작한다.

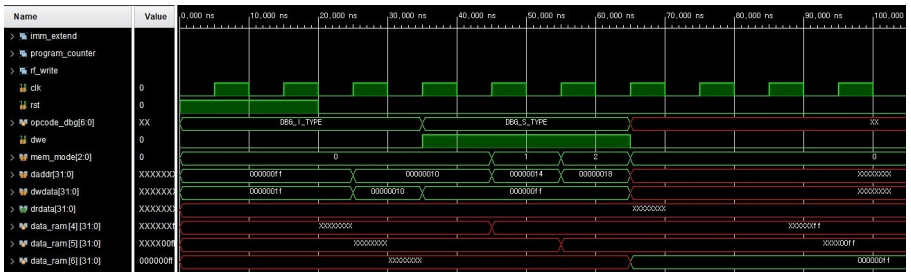
[코드] data_mem.sv — SB/SH/SW 쓰기 로직 (posedge clk 동기)

```
// ── 메모리 쓰기 (posedge clk, dwe=1 일 때)
always_ff @(posedge clk) begin
    if (dwe) begin
        case (mem_mode)
            `SB: case (daddr[1:0])
                2'b00: data_ram[daddr[31:2]][7:0]   <= dwdata[7:0];
                2'b01: data_ram[daddr[31:2]][15:8]  <= dwdata[7:0];
                2'b10: data_ram[daddr[31:2]][23:16] <= dwdata[7:0];
                2'b11: data_ram[daddr[31:2]][31:24] <= dwdata[7:0];
            endcase
            `SH: begin
                if (daddr[1] == 0) data_ram[daddr[31:2]][15:0]  <= dwdata[15:0];
                else               data_ram[daddr[31:2]][31:16] <= dwdata[15:0];
            end
```

```
            `SW: data_ram[daddr[31:2]] <= dwdata;
        endcase
    end
end
```

[코드] data_mem.sv — LB/LH/LW/LBU/LHU 읽기 로직 (비동기 조합)

```
// ── 메모리 읽기 (always_comb, 비동기)
always_comb begin
    drdata = 0;
    case (mem_mode)
        `LB: case (daddr[1:0])   // 부호확장 byte load
            2'b00: drdata = {{24{data_ram[daddr[31:2]][7]}},  data_ram[daddr[31:2]][7:0]};
            2'b01: drdata = {{24{data_ram[daddr[31:2]][15]}}, data_ram[daddr[31:2]][15:8]};
            2'b10: drdata = {{24{data_ram[daddr[31:2]][23]}}, data_ram[daddr[31:2]][23:16]};
            2'b11: drdata = {{24{data_ram[daddr[31:2]][31]}}, data_ram[daddr[31:2]][31:24]};
        endcase
        `LH: begin               // 부호확장 halfword load
            if (daddr[1] == 0) drdata = {{16{data_ram[daddr[31:2]][15]}},
data_ram[daddr[31:2]][15:0]};
            else               drdata = {{16{data_ram[daddr[31:2]][31]}},
data_ram[daddr[31:2]][31:16]};
        end
        `LW:  drdata = data_ram[daddr[31:2]];
        `LBU: case (daddr[1:0]) // 0확장 byte load
            2'b00: drdata = {{24{1'b0}}, data_ram[daddr[31:2]][7:0]};
            2'b01: drdata = {{24{1'b0}}, data_ram[daddr[31:2]][15:8]};
            ...
        endcase
        `LHU: begin              // 0확장 halfword load
            if (daddr[1] == 0) drdata = {{16{1'b0}}, data_ram[daddr[31:2]][15:0]};
            else               drdata = {{16{1'b0}}, data_ram[daddr[31:2]][31:16]};
        end
    endcase
end
```



[그림] data_mem 검증 파형 — SB(mem_mode=0): data_ram[4][7:0]=0xFF, SH(1): data_ram[5][15:0]=0x00FF, SW(2): data_ram[6]=0x000000FF 순서대로 byte-enable 쓰기 확인

3.8 Testbench

tb_rv32i 는 10ns 주기(50MHz) 클럭을 생성하고, negedge 기준 2 사이클 리셋 후 20 사이클 자유 실행한다. $readmemh 기반으로 instruction_code.mem 또는 instruction_mem_sort.mem 을 교체 로드하여 다양한 테스트를 수행한다.

[코드] tb_rv32i.sv — 전체 테스트벤치

```
module tb_rv32i ();
    logic clk, rst;
    top_rv32i_soc dut (.*)   // 전체 SoC 인스턴스

    always #5 clk = ~clk;   // 10ns 주기 (50MHz) 클럭

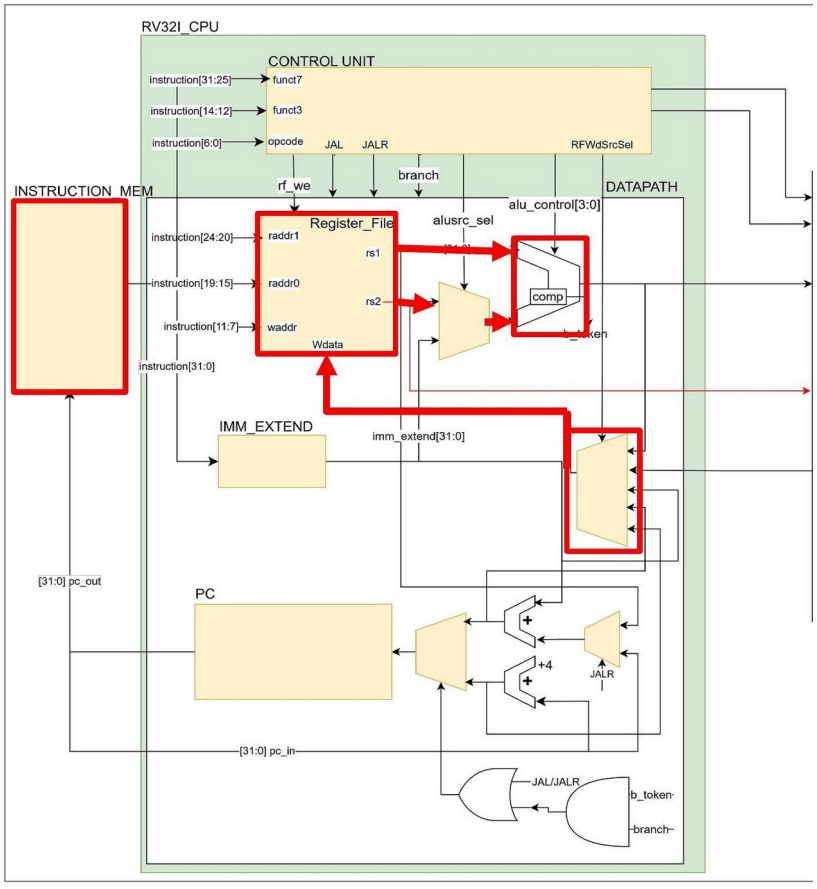
    initial begin
        clk = 0;
        rst = 1;             // 리셋 활성화
        @(negedge clk);
        @(negedge clk);      // 2사이클 리셋 유지
        rst = 0;             // 리셋 해제 → 명령어 실행 시작
        repeat(20) @(negedge clk);  // 20사이클 실행
        $stop;
    end
endmodule
```

4. 명령어 타입별 상세 동작 및 시뮬레이션

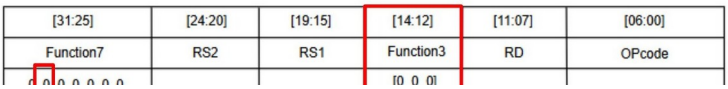
4.1 R-TYPE (Register)

R-Type 명령어는 rs1, rs2 두 레지스터를 읽어 ALU 연산 후 결과를 rd 에 저장한다. Control Unit 은 rf_we=1, alusrc_sel=0(rs2 선택), rfsrc_sel=000(ALU 결과)으로 설정한다. 즉시값을 사용하지 않으므로 imm_extend 출력은 무시된다.

alu_control = {funct7[5], funct3} 조합으로 10 가지 연산을 구분한다. ADD 와 SUB 는 funct3=000 으로 동일하지만 funct7[5]로 구분(0=ADD, 1=SUB)하며, SRL 과 SRA 도 같은 방식으로 구분(0=SRL, 1=SRA)한다.

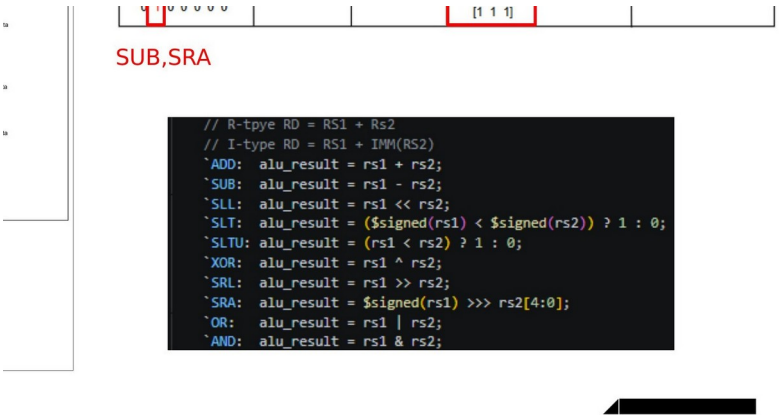


[그림] R-TYPE 데이터 흐름 B/D — 빨간선: rs1/rs2→ALU→WB MUX→register_file

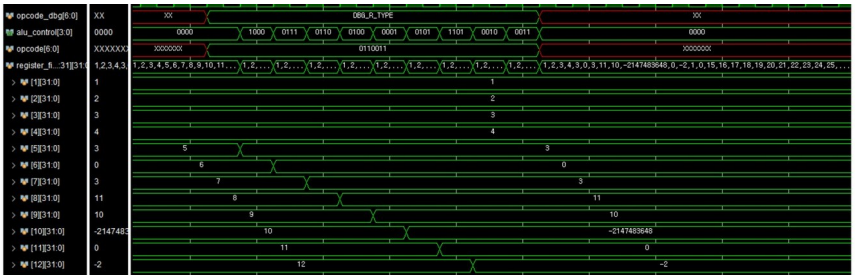


[31:25]	[24:20]	[19:15]	[14:12]	[11:07]	[06:00]
Function7	RS2	RS1	Function3	RD	OPcode

[그림] R-TYPE 명령어 포맷 (funct7/rs2/rs1/funct3/rd/opcode)



[그림] R-TYPE ALU 연산 코드 (ADD~AND 10 종)



[그림] R-TYPE 시뮬레이션 파형 — alu_control 변화 및 register[1]~[14] 누적 결과

파형 분석: opcode_dbg 가 DBG_R_TYPE 으로 표시되는 구간에서 alu_control 이 1000(ADD)→0111(AND)→0110(OR)→... 순으로 변화하며, 각 명령어 실행 후 해당 레지스터에 결과가 기록된다. x5(ADD)=1+2=3, x6(SUB)=3-3=0, x7(AND)=7&3=3, x8(OR)=8|3=11 등 모든 결과가 정상 확인되었다.

```
// R-type simulation
    instr_rom[1]  = 32'h002082B3;  // add  x5,  x1,  x2
    instr_rom[2]  = 32'h40328333;  // sub  x6,  x5,  x3
    instr_rom[3]  = 32'h0033F3B3;  // and  x7,  x7,  x3
    instr_rom[4]  = 32'h00346433;  // or   x8,  x8,  x3
    instr_rom[5]  = 32'h0034C4B3;  // xor  x9,  x9,  x3
    instr_rom[6]  = 32'h01F41533;  // sll  x10, x8,  x3
    instr_rom[7]  = 32'h01E4D5B3;  // srl  x11, x9,  x3
    instr_rom[8]  = 32'h41E55633;  // sra  x12, x10, x3
    instr_rom[9]  = 32'h0041A6B3;  // slt  x13, x3,  x4
    instr_rom[10] = 32'h00323733;  // sltu x14, x4,  x3
```

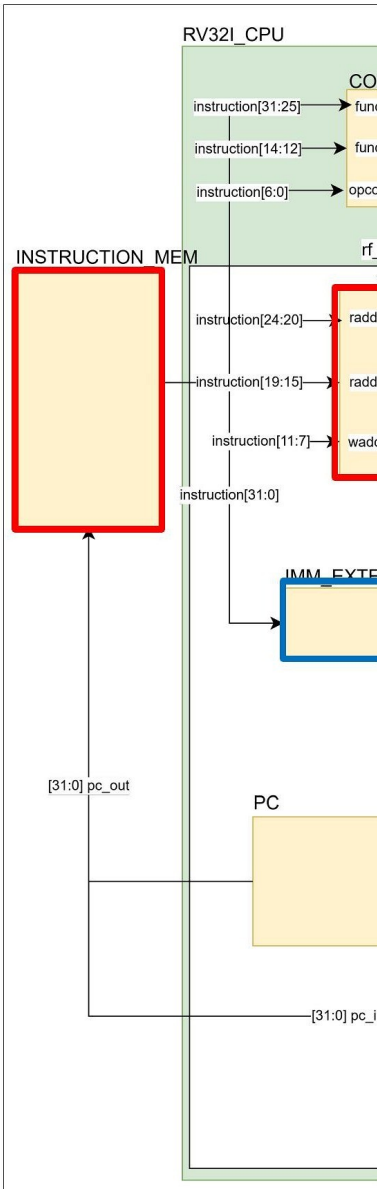
[그림] R-TYPE 기계어 코드 — instruction_code.mem 인코딩

PC	Instruction	rs1	rs2	rd	Result
0	add x5,x1,x2	1	2	5	x5=3
4	sub x6,x2,x1	2	1	6	x6=1
8	and x7,x3,x1	3	1	7	x7=1
12	or x8,x3,x1	3	1	8	x8=3
16	xor x9,x3,x1	3	1	9	x9=2
20	sll x10,x1,x2	1	2	10	x10=4
24	srl x11,x8,x1	8	1	11	x11=4
28	sra x12,x13,x1	13	1	12	x12=-2147483642
32	slt x13,x1,x2	1	2	13	x13=1
36	sltu x14,x1,x2	1	2	14	x14=1

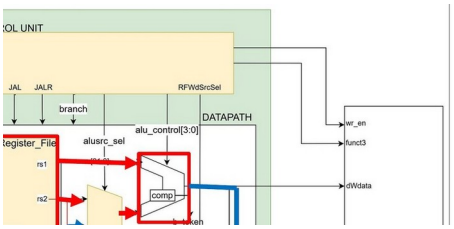
[그림] R-TYPE 검증 결과 — PC / Instruction / rs1 / rs2 / rd / Result

4.2 I-TYPE (Immediate Arithmetic)

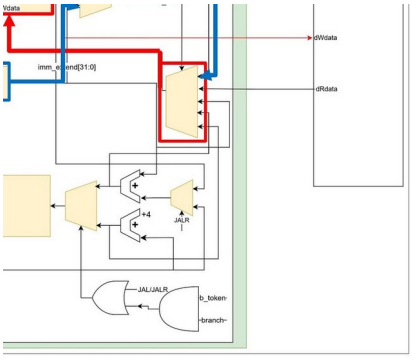
I-Type 산술 명령어는 rs1 과 12 비트 부호확장 즉시값을 ALU 에 입력하여 연산한다. alusrc_sel=1 로 MUX 가 rs2 대신 imm_extend 출력을 ALU 두 번째 입력으로 선택한다. SRLI 와 SRAI 는 funct3=101 로 동일하므로 funct7[5] 로 구분이 필요하며, 나머지 I-Type 은 funct7=0 으로 고정된다.



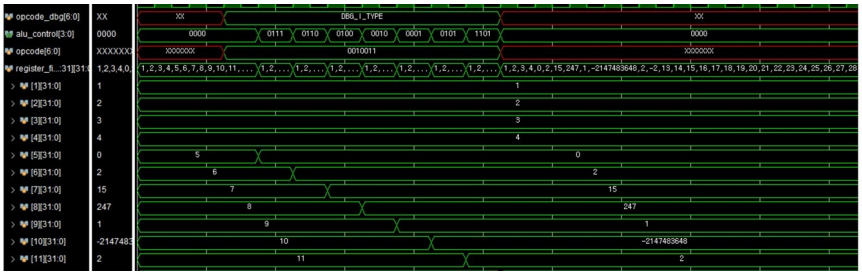
[그림] I-TYPE 데이터 흐름 B/D — 파란선: imm_extend→MUX→ALU



[그림] I-TYPE 명령어 포맷 및 Shift 명령어(SLLI/SRLI/SRAI) 포맷



[그림] I-TYPE ALU 연산 코드 (ADDI~SRAI)



[그림] I-TYPE 시뮬레이션 파형 — register[5]~[12] 결과 확인

파형 분석: X5(ADDI)=1+(-1)=0, X6(ANDI)=2&15=2, X7(ORI)=2|15=15, X8(XORI)=8^255=247, X9(SLTI)=3<5=1, X10(SLLI)=1<<31=-2147483648, X11(SRLI)=X10>>30=2, X12(SRAI)=X10>>30=-2 (부호 유지) 모두 정상 동작 확인.

```
// I-type simulation
instr_rom[1] = 32'hFFF08293;  // addi x5,  x1, -1
instr_rom[2] = 32'h00F17313;  // andi x6,  x2, 15
instr_rom[3] = 32'h00F16393;  // ori  x7,  x2, 15
instr_rom[4] = 32'h0FF44413;  // xori x8,  x8, 25
instr_rom[5] = 32'h0051A493;  // slti x9,  x3, 5
// 1 << 31 = 0x80000000
instr_rom[6] = 32'h01F09513;  // slli x10, x1, 31
// 0x80000000 >> 30 = 0x00000002
instr_rom[7] = 32'h01E55593;  // srli x11, x10, 3
// 0x80000000 >>> 30 = 0xFFFFFFFE
```

[그림] I-TYPE 기계어 코드 — instruction_code.mem 인코딩

값

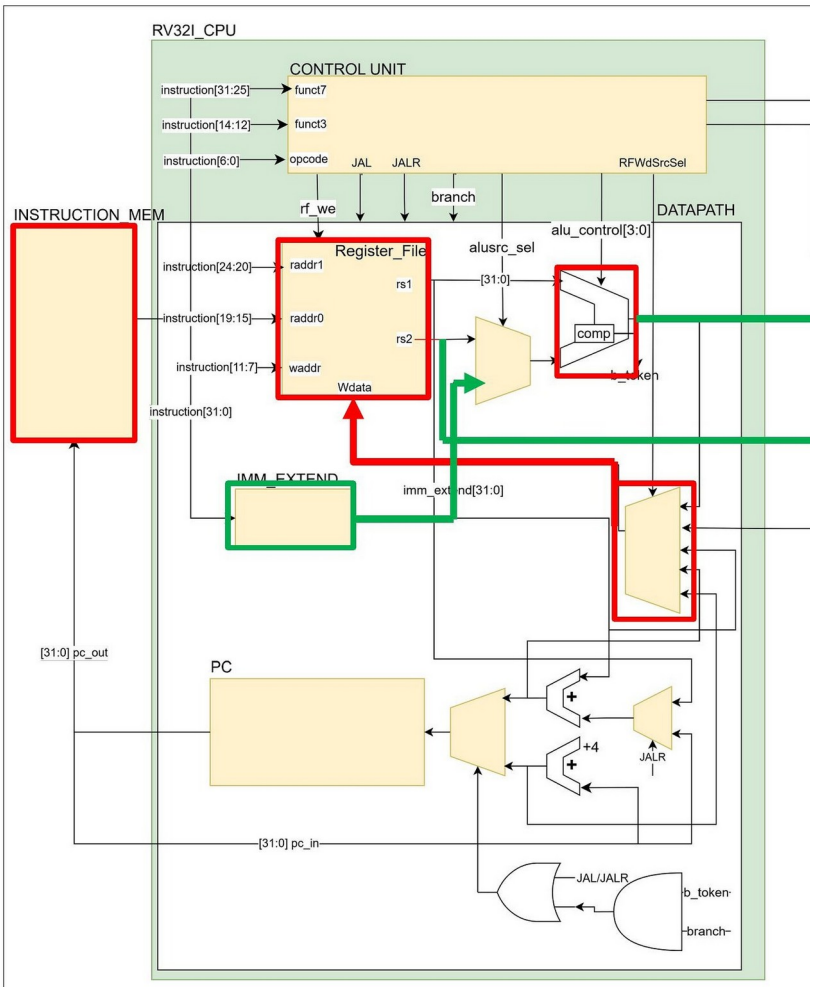
PC	Instr	계산	결과
4	addi x5,x1,-1	1 + (-1)	x5 = 0
8	andi x6,x2,15	2 & 15	x6 = 2
12	ori x7,x2,15	2 | 15	x7 = 15
16	xori x8,x8,255	8 ^ 255	x8 = 247
20	slti x9,x3,5	3 < 5	x9 = 1
24	slli x10,x1,31	1 << 31	x10 = 0x80000000
28	srli x11,x10,30	0x80000000 >> 30	x11 = 0x00000002
32	srai x12,x10,30	arithmetic shift	x12 = 0xFFFFFFFE

[그림] I-TYPE 검증 결과 — PC / Instruction / 계산식 / Result

4.3 S-TYPE (Store) & L-TYPE (Load)

S-Type 은 메모리에 데이터를 저장한다. rs1+imm_extend 로 쓰기 주소를 계산(ALU 의 ADD 연산)하고, rs2 값을 dwdata 로 data_mem 에 전달한다. dwe=1 로 메모리 쓰기를 활성화하며 rf_we=0 으로 레지스터 파일에는 아무것도 쓰지 않는다. dwdata 경로는 다이어그램에서 빨간 선으로 표시된 rs2 직접 연결 경로이다.

L-Type 은 rs1+imm 으로 메모리 주소를 계산하고 해당 위치의 데이터를 읽어 rd 에 저장한다. rfsrc_sel=001 로 WB MUX 가 drdata 를 선택하여 레지스터에 기록한다. LB/LH 는 부호확장(MSB 복제), LBU/LHU 는 상위 비트를 0 으로 채워 반환한다.



[그림] S-TYPE 데이터 흐름 B/D — 초록선: rs2→dwdata, imm→ALU 주소 계산

Store data into RAM

[31:25]	[24:20]	[19:15]	[14:12]	[11:07]	[06:00]
Imm[11:5]	RS2	RS1	Function3	Imm[4:1][11]	OPcode
Byte Half Whole			[0 0 0] [0 0 1] [0 1 0]		0 1 0 0 0 1 1

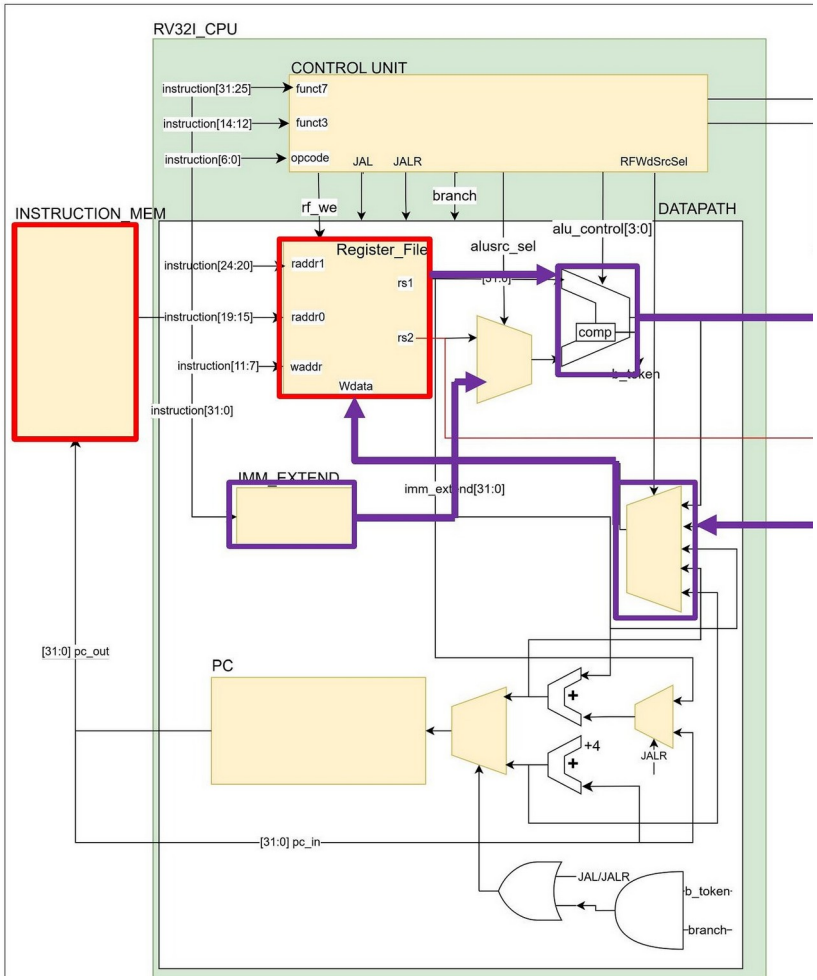
[그림] S-TYPE 명령어 포맷 (imm[11:5]/rs2/rs1/funct3/imm[4:0]/opcode)

Byte	Addr : rs1 + Imm	Data : wData[0:7]
Half	Addr : rs1 + Imm	Data : wData[0:15]

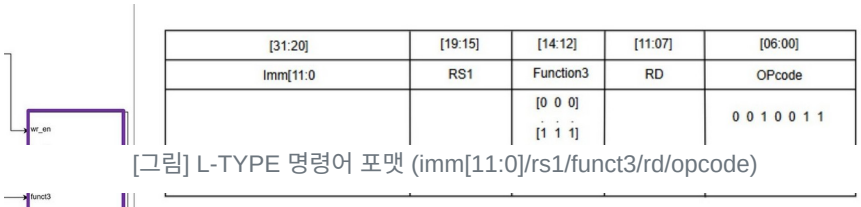
[그림] S-TYPE 저장 크기별 동작 (Byte/Half/Whole)

```
`SB:
case (daddr[1:0])
    2'b00: data_ram[daddr[31:2]][7:0] <= dwdata[7:0];
    2'b01: data_ram[daddr[31:2]][15:8] <= dwdata[7:0];
    2'b10: data_ram[daddr[31:2]][23:16] <= dwdata[7:0];
    2'b11: data_ram[daddr[31:2]][31:24] <= dwdata[7:0];
endcase
`SH: begin
    if (daddr[1] == 0)
        data_ram[daddr[31:2]][15:0] <= dwdata[15:0];
    else data_ram[daddr[31:2]][31:16] <= dwdata[15:0];
end
`SW: data_ram[daddr[31:2]] <= dwdata;
```

[그림] S-TYPE 메모리 쓰기 코드 (SB/SH/SW byte-enable 처리)

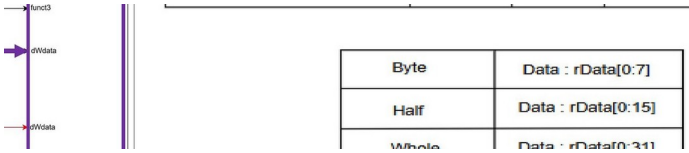


[그림] L-TYPE 데이터 흐름 B/D — 보라선: drdata→WB MUX→register_file



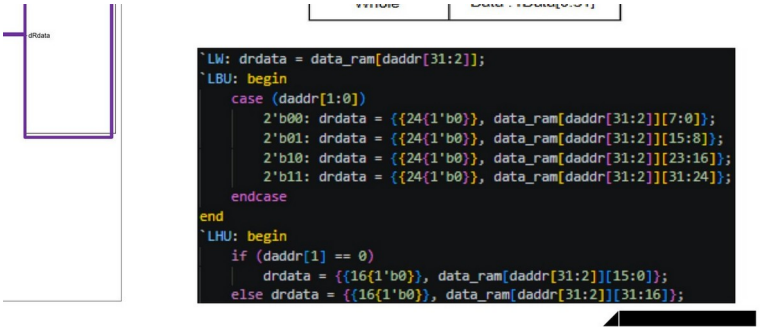
[31:20]	[19:15]	[14:12]	[11:07]	[06:00]
Imm[11:0	RS1	Function3	RD	OPcode
		[0 0 0] . . . [1 1 1]		0 0 1 0 0 1 1

[그림] L-TYPE 명령어 포맷 (imm[11:0]/rs1/funct3/rd/opcode)



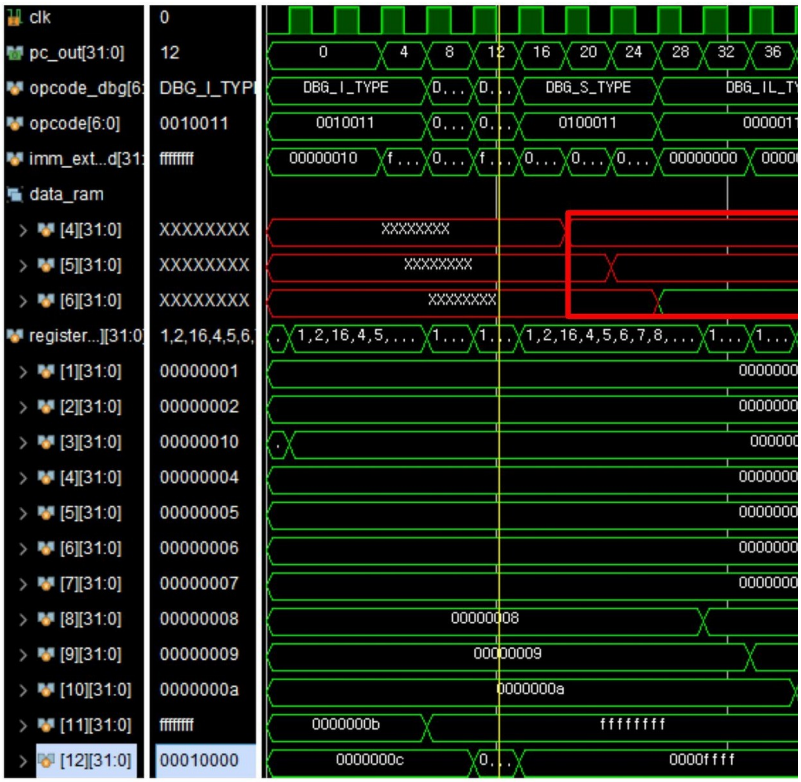
Byte	Data : rData[0:7]
Half	Data : rData[0:15]
Whole	Data : rData[0:31]

[그림] L-TYPE 읽기 크기별 동작 (Byte/Half/Whole)

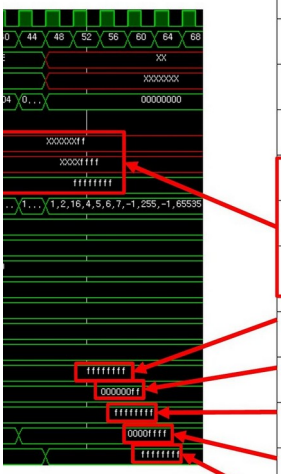


```
`LW: drdata = data_ram[daddr[31:2]];
`LBU: begin
    case (daddr[1:0])
        2'b00: drdata = {{24{1'b0}}, data_ram[daddr[31:2]][7:0]};
        2'b01: drdata = {{24{1'b0}}, data_ram[daddr[31:2]][15:8]};
        2'b10: drdata = {{24{1'b0}}, data_ram[daddr[31:2]][23:16]};
        2'b11: drdata = {{24{1'b0}}, data_ram[daddr[31:2]][31:24]};
    endcase
end
`LHU: begin
    if (daddr[1] == 0)
        drdata = {{16{1'b0}}, data_ram[daddr[31:2]][15:0]};
    else drdata = {{16{1'b0}}, data_ram[daddr[31:2]][31:16]};
```

[그림] L-TYPE 메모리 읽기 코드 (LB/LBU/LH/LHU/LW 부호확장)



[그림] S & L-TYPE 시뮬레이션 파형 — data_ram 내부 값 변화, drdata 부호확장 확인



PC	Instruction	동작	결과
0	addi x3,x0,16	base 주소 설정	x3=0x10
1	addi x11,x0,-1	-1 생성	x11=FFFFFFFF
2	lui x12,0x10	upper 생성	x12=00010000
3	addi x12,x12,-1	하위값 조정	x12=0000FFFF
4	sb x11,0(x3)	byte 저장	MEM[0x10]=FF
5	sh x12,4(x3)	half 저장	MEM[0x14]=FFFF
6	sw x11,8(x3)	word 저장	MEM[0x18]=FFFFFFFF
7	lb x8,0(x3)	signed byte load	x8=FFFFFFFF
8	lbu x9,0(x3)	unsigned byte load	x9=000000FF
9	lh x10,4(x3)	signed half load	x10=FFFFFFFF
10	lhu x11,4(x3)	unsigned half load	x11=0000FFFF
11	lw x12,8(x3)	word load	x12=FFFFFFFF

[그림] S & L-TYPE 검증 결과 — PC / Instruction / 동작 / 결과

파형 분석: addi 로 기준 주소(x3=0x10)와 데이터 준비 후 sb x11,0(x3)으로 data_ram[4][7:0]=0xFF 저장, sh x12,4(x3)으로 data_ram[5][15:0]=0xFFFF 저장, sw x11,8(x3)으로 data_ram[6]=0xFFFFFFFF 저장 확인. lb 로 읽으면 x8=0xFFFFFFFF(부호확장), lbu 로 읽으면 x9=0x000000FF(0 확장) 확인.

```
// base
instr_rom[0] = 32'h01000193; // addi x3,x0,16

// data
instr_rom[1]  = 32'hFFF00593;  // addi x11,x0,-1
instr_rom[2]  = 32'h00010637;  // lui x12,0x10
instr_rom[3]  = 32'hFFF60613;  // addi x12,x12,-

// store
instr_rom[4]  = 32'h00B18023;  // sb x11,0(x3)
instr_rom[5]  = 32'h00C19223;  // sh x12,4(x3)
instr_rom[6]  = 32'h00B1A423;  // sw x11,8(x3)

// load
instr_rom[7]  = 32'h00018403;  // lb  x8,0(x3)
instr_rom[8]  = 32'h0001C483;  // lbu x9,0(x3)
instr_rom[9]  = 32'h00419503;  // lh  x10,4(x3)
instr_rom[10] = 32'h0041D583;  // lhu x11,4(x3)
instr_rom[11] = 32'h0081A603;  // lw  x12,8(x3)
```

[그림] S & L-TYPE 기계어 코드 — instruction_code.mem 인코딩 (SB/SH/SW/LB/LBU/LH/LHU/LW)

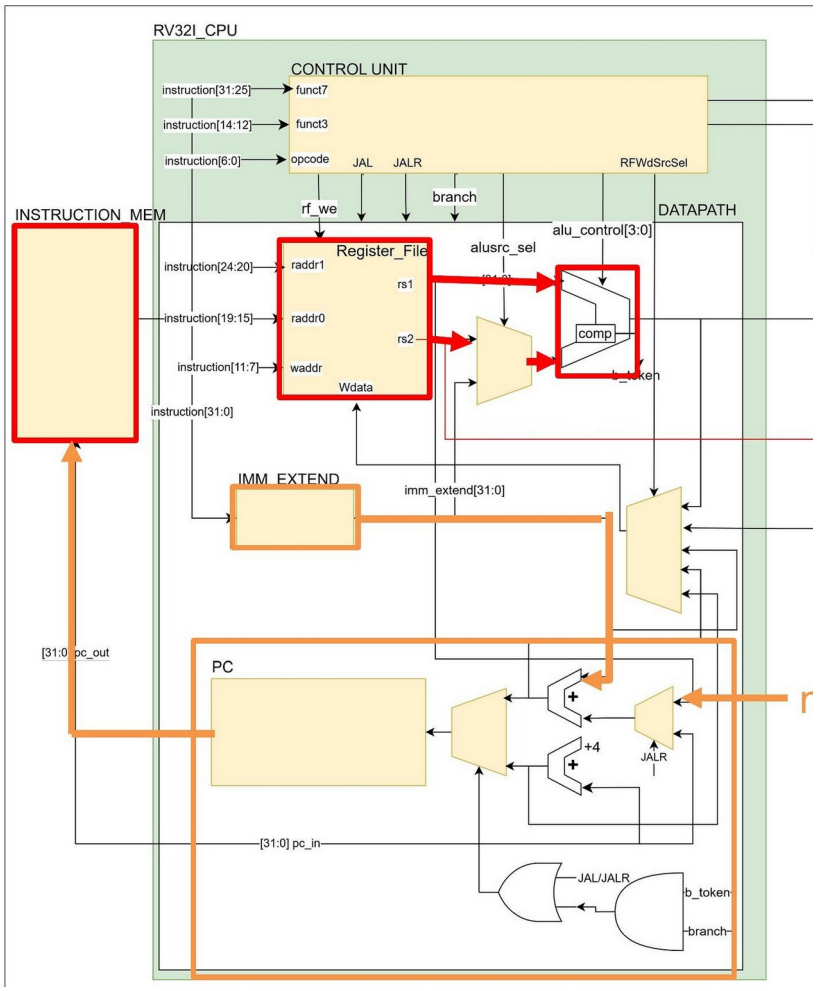
PC	Instruction	동작	결과
0	addi x3,x0,16	base 주소 설정	x3=0x10
1	addi x11,x0,-1	-1 생성	x11=FFFFFFFF
2	lui x12,0x10	upper 생성	x12=00010000
3	addi x12,x12,-1	하위값 조정	x12=0000FFFF
4	sb x11,0(x3)	byte 저장	MEM[0x10]=FF
5	sh x12,4(x3)	half 저장	MEM[0x14]=FFFF
6	sw x11,8(x3)	word 저장	MEM[0x18]=FFFFFFFF
7	lb x8,0(x3)	signed byte load	x8=FFFFFFFF
8	lbu x9,0(x3)	unsigned byte load	x9=000000FF
9	lh x10,4(x3)	signed half load	x10=FFFFFFFF
10	lhu x11,4(x3)	unsigned half load	x11=0000FFFF
11	lw x12,8(x3)	word load	x12=FFFFFFFF

[그림] S & L-TYPE 검증 결과 — PC / Instruction / 동작 / 결과 전체 표

4.4 B-TYPE (Branch)

B-Type 은 rs1 과 rs2 를 비교하여 조건 성립 시 PC 를 imm 오프셋만큼 이동시킨다. b_taken 신호가 1 이면 SRC MUX 가 pc_imm(=PC+imm)을 다음 PC 로 선택하고, 0 이면 pc_4(=PC+4)를 선택한다. branch=1 이 활성화된 상태에서만 b_taken 이 PC 변경에 영향을 준다.

B-Type 의 즉시값은 imm[12:1]로 인코딩되며(imm[0]=0, 2 바이트 정렬), 비트가 분산되어 있어 imm_extend 에서 재조합한다. 분기 범위는 ±4KB(±2^12 바이트)이다.

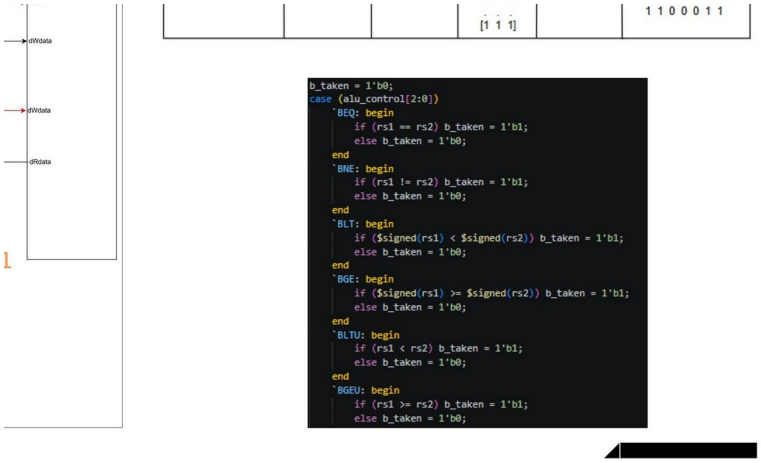


[그림] B-TYPE 데이터 흐름 B/D — 주황선: b_taken→PC SRC MUX, branch 신호 경로

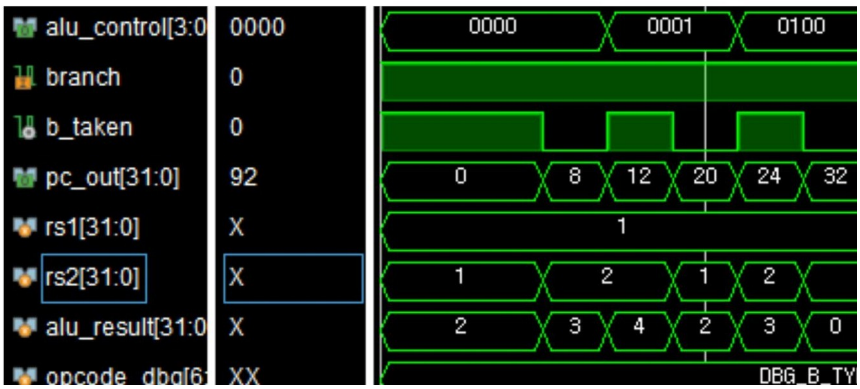
Compare Data & Make Branch (PC += Imm)

[31:25]	[24:20]	[19:15]	[14:12]	[11:07]	[06:00]
Imm[12][10:5]	RS2	RS1	Function3	Imm[4:1][11]	OPcode
			[0 0 0]		

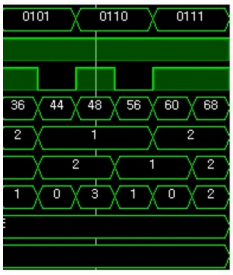
[그림] B-TYPE 명령어 포맷 (imm[12,10:5]/rs2/rs1/funct3/imm[4:1,11]/opcode)



[그림] B-TYPE b_taken 생성 코드 (if-else 수정 후 버전)



[그림] B-TYPE 시뮬레이션 파형 — b_taken 토글 및 pc_out 분기 동작 확인



PC	Instruction	Condition	Branch	Next PC
0	beq x1,x1	1 == 1	taken	8
8	beq x1,x2	1 == 2	not taken	12
12	bne x1,x2	1 != 2	taken	20
20	bne x1,x1	1 != 1	not taken	24
24	blt x1,x2	1 < 2	taken	32
32	blt x2,x1	2 < 1	not taken	36
36	bge x2,x1	2 >= 1	taken	44
44	bge x1,x2	1 >= 2	not taken	48
48	bltu x1,x2	1 < 2	taken	56
56	bltu x2,x1	2 < 1	not taken	60
60	bgeu x2,x1	2 >= 1	[redacted]	[redacted]

[그림] B-TYPE 검증 결과 — PC / Instruction / Condition / Branch / Next PC

파형 분석: PC=0 에서 beq x1,x1(1==1) → taken → pc=8, PC=8 에서 beq x1,x2(1==2) → not taken → pc=12, PC=12 에서 bne x1,x2(1≠2) → taken → pc=20, PC=24 에서 blt x1,x2(1<2) → taken → pc=32 등 BEQ~BGEU 6 종 전체 정상 확인.

```
//// B-type
// beq
instr_rom[0]  = 32'h00108463;  // beq  x1,x1,+8   taken
instr_rom[2]  = 32'h00208463;  // beq  x1,x2,+8   not taken

// bne
instr_rom[3]  = 32'h00209463;  // bne  x1,x2,+8   taken
instr_rom[5]  = 32'h00109463;  // bne  x1,x1,+8   not taken

// blt
instr_rom[6]  = 32'h0020C463;  // blt  x1,x2,+8   taken
instr_rom[8]  = 32'h0010C463;  // blt  x2,x1,+8   not taken

// bge
instr_rom[9]  = 32'h00115463;  // bge  x2,x1,+8   taken
instr_rom[11] = 32'h00215463;  // bge  x1,x2,+8   not taken

// bltu
instr_rom[12] = 32'h0020E463;  // bltu x1,x2,+8   taken
instr_rom[14] = 32'h0010E463;  // bltu x2,x1,+8   not taken

// bgeu
instr_rom[15] = 32'h00117463;  // bgeu x2,x1,+8   taken
instr_rom[17] = 32'h00217463;  // bgeu x1,x2,+8   not taken
```

[그림] B-TYPE 기계어 코드 — BEQ/BNE/BLT/BGE/BLTU/BGEU 각 taken/not taken

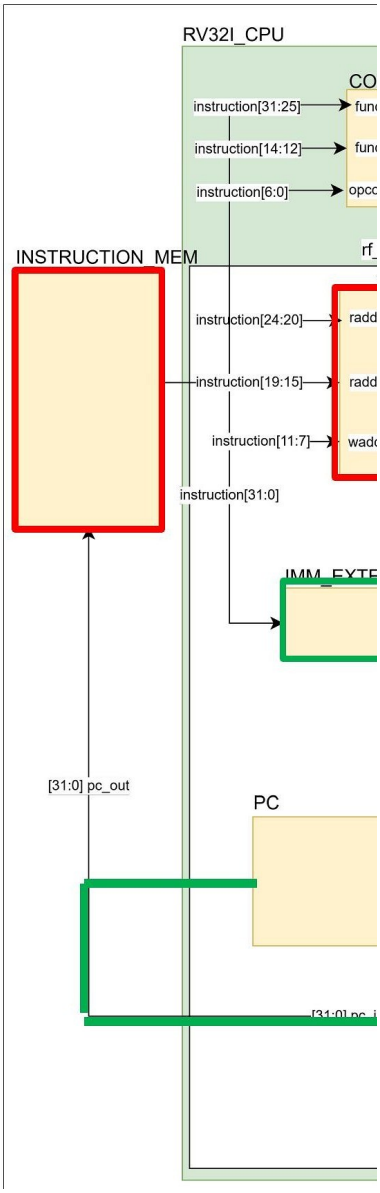
PC	Instruction	Condition	Branch	Next PC
0	beq x1,x1	1 == 1	taken	8
8	beq x1,x2	1 == 2	not taken	12
12	bne x1,x2	1 != 2	taken	20
20	bne x1,x1	1 != 1	not taken	24
24	blt x1,x2	1 < 2	taken	32
32	blt x2,x1	2 < 1	not taken	36
36	bge x2,x1	2 >= 1	taken	44
44	bge x1,x2	1 >= 2	not taken	48
48	bltu x1,x2	1 < 2	taken	56
56	bltu x2,x1	2 < 1	not taken	60

[그림] B-TYPE 검증 결과 — PC / Instruction / Condition / Branch / Next PC

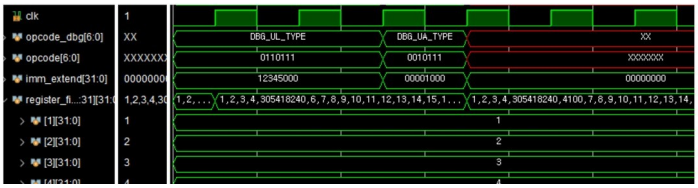
4.5 UL-TYPE (LUI) & UA-TYPE (AUIPC)

LUI(Load Upper Immediate)는 20 비트 즉시값을 레지스터 상위 20 비트에 배치하고 하위 12 비트는 0 으로 채운다. 이후 ADDI 로 하위 비트를 채우는 2 단계 방식으로 32 비트 상수를 구성할 때 사용된다. imm_extend = {instr_code[31:12], 12'h000}으로 생성되며, rfsrc_sel=010 으로 WB MUX 가 imm_extend 자체를 rd 에 직접 기록한다.

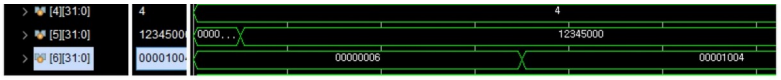
AUIPC(Add Upper Immediate to PC)는 LUI 와 동일한 포맷이지만 PC + (imm<<12) 결과를 rd 에 저장한다. rfsrc_sel=011 로 WB MUX 가 pc_imm(=PC + imm_extend)를 선택한다. 위치 독립 코드(PIC)나 전역 변수 접근 등 PC 상대 절대 주소 계산에 활용된다.



[그림] UA-TYPE 데이터 흐름 B/D — 초록선: imm_extend+PC→pc_imm→register_file



[그림] UL & UA-TYPE 시뮬레이션 파형 — LUI x5=0x12345000, AUIPC x6=0x00001004



PC	Instr	Type	Operation	Result
0	lui x5,0x12345	LUI	x5 = 0x12345000	x5 = 0x12345000

[그림] UL & UA-TYPE 검증 결과 — PC / Instr / Type / Operation / Result



[그림] UL/UA-TYPE 기계어 코드

PC	Instr	Type	Operation	Result
4	auipc x6,0x1	AUIPC	x6 = PC + 0x1000	x6 = 0x00001004

[그림] UL/UA-TYPE 검증 결과 표



LUI
Address Shift / Jump (imm << 12)

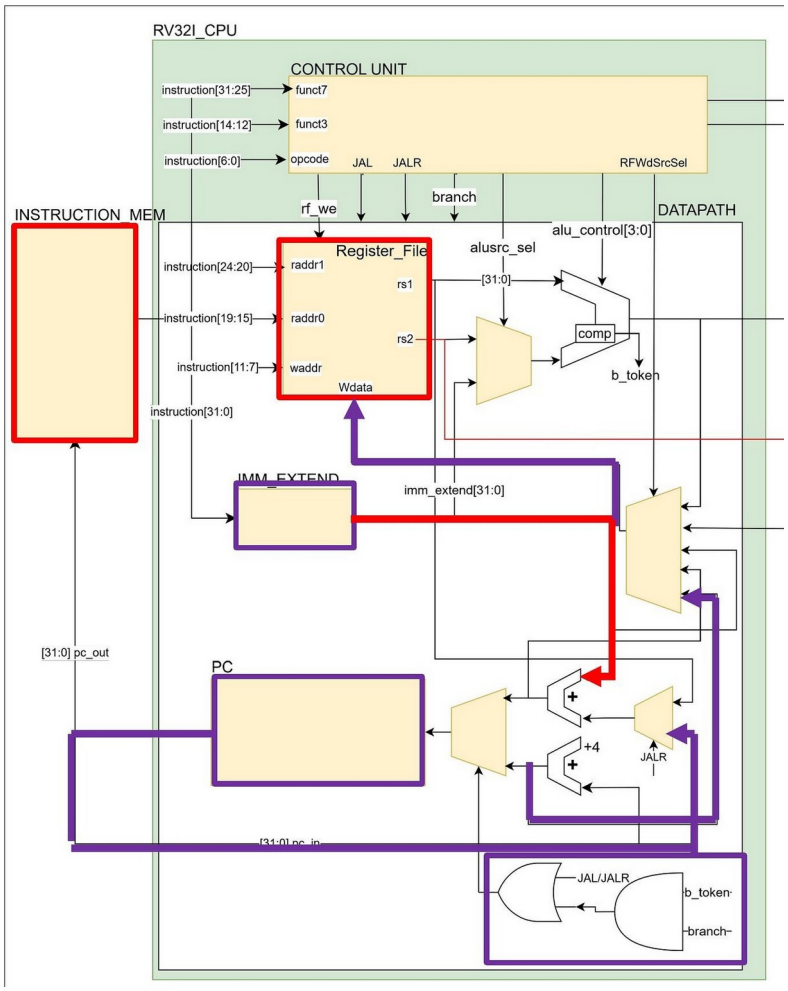
AUIPC
Address Shift / Jump PC + (imm << 12)

[그림] LUI / AUIPC 명령어 포맷 비교

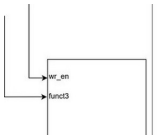
4.6 J-TYPE (JAL) & JL-TYPE (JALR)

JAL(Jump and Link)은 현재 PC 에 20 비트 부호확장 오프셋을 더한 주소로 점프하고, PC+4 를 rd 에 저장한다. jal=1 이 활성화되면 PC SRC MUX 가 pc_imm 을 다음 PC 로 선택하며, rfsrc_sel=100 으로 WB MUX 가 pc_4 를 rd 에 기록한다. 점프 범위는 ±1MB(±2^20 바이트)이다.

JALR(Jump and Link Register)은 I-Type 으로 분류되지만 동작은 JAL 과 유사하다. jalr=1 이 활성화되면 PC JALR MUX 가 rs1 을 선택하여 rs1+imm 절대 주소로 점프한다. 함수 포인터 호출, 인터럽트 복귀(ret: jalr x0, ra, 0), 동적 디스패치 등에 활용된다.



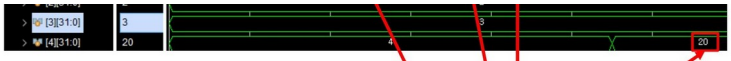
[그림] J-TYPE 데이터 흐름 B/D — 보라선: pc_imm(PC+imm)→PC, pc_4→rd



[그림] J-TYPE 명령어 포맷 (imm[20,10:1,11,19:12]/rd/opcode)



[그림] J & JL-TYPE 시뮬레이션 파형 — jal x1+8 점프, jalr x4,0(x4) 복귀 동작



PC	Instr	Type	rd	wb_out	PC next	설명
0	jal x1,+8	JAL	x1	4	8	jump
8	nop	-	-	-	12	Nop

[그림] J & JL-TYPE 검증 결과 — PC / Instr / Type / rd / wb_out / PC_next

파형 분석: PC=0 에서 jal x1,+8 실행 → pc_next=8, x1=4(복귀 주소 저장), PC=8,12 에서 nop 실행, PC=16 에서 jalr x2,0(x4) 실행(x4=4) → pc_next=4 로 복귀, x2=20(복귀 주소) 정상 확인.

```
// J-type
instr_rom[0] = 32'h008000EF;   // jal  x1
```

[그림] J/JL-TYPE 기계어 코드

```
1, +8


t, 0(x4)
```

PC	Instr	Type	rd	wb_out	PC next	설명
0	jal x1,+8	JAL	x1	4	8	jump
8	nop	-	-	-	12	Nop
12	nop	-	-	-	16	nop
16	jalr x2,0(x4)	JALR	x4	20	4	return

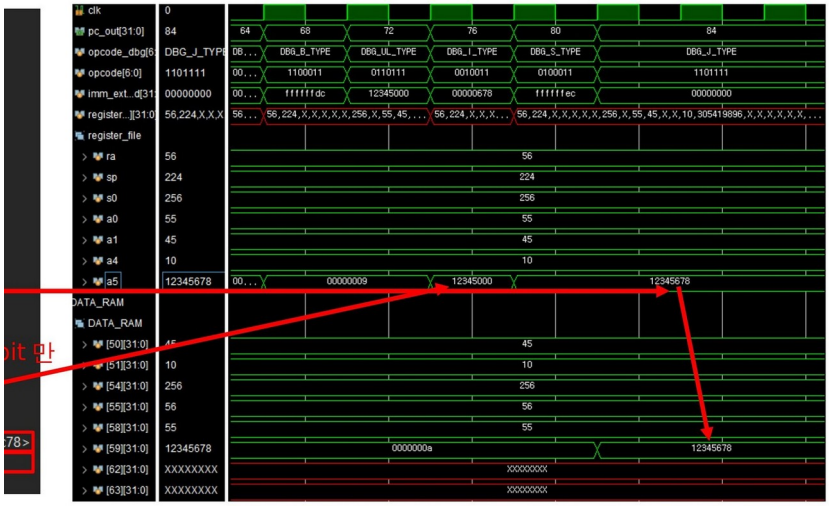
[그림] J/JL-TYPE 검증 결과 — PC / Instr / Type / rd / wb_out / PC_next / 설명

```
0000000000000000 :
   0:  10000113          li      sp,256

0000000000000004
:(DEC)
   4:  fe010113          addi    sp,sp,-32
   8:  00112e23          sw      ra,28(sp)
  12:  00812c23          sw      s0,24(sp)
  16:  02010413          addi    s0,sp,32
  20:  fe042623          sw      zero,-20(s0)
  24:  fe042423          sw      zero,-24(s0)
  28:  0200006f          j       3c
  32:  fec42783          lw      a5,-20(s0)
  36:  00178793          addi    a5,a5,1
  40:  fef42623          sw      a5,-20(s0)
  44:  fe842583          lw      a1,-24(s0)
  48:  fec42503          lw      a0,-20(s0)
  52:  024000ef          jal     ra,58
  56:  fea42423          sw      a0,-24(s0)
  60:  fec42703          lw      a4,-20(s0)
  64:  00900793          li      a5,9
  68:  fce7dee3          ble     a4,a5,20 (DEC:32)
  72:  123457b7          lui     a5,0x12345
  76:  67878793          addi    a5,a5,1656 # 12345678 <_sstack+0x1233
```

LUI 의 Imm 은 20
가능하기 때문

[그림] LUI 20 비트 제한 — LUI+ADDI 2 단계 방식 어셈블리 코드



[그림] LUI+ADDI 파형 — a5 에 0x12345678 구성 과정 (LUI 로 상위, ADDI 로 하위 합산)

5. 통합 검증 — 1to10 Adder 프로그램

5.1 개요

1 부터 10 까지의 합산을 수행하는 프로그램을 C 언어 동작 기준으로 작성하고, 이를 RISC-V RV32I 어셈블리로 변환하여 실행하였다. 이 과정에서 R/I/S/L/B/J/JL 등 대부분의 명령어 타입이 실제로 사용되며, 스택 프레임 관리, 반복문, 함수 호출·복귀까지 종합적으로 검증된다.

5.2 C 코드 → 어셈블리 변환 과정

아래는 C 언어로 작성된 원본 코드와, 이를 GCC RISC-V 크로스컴파일러로 변환한 어셈블리의 대응 관계다. while 루프 조건(a<10)이 BLE 분기 명령어로, 함수 호출(adder)이 JAL+JALR 로, 반환값(a+b)이 ADD 명령어로 매핑됨을 확인할 수 있다.

```c
int adder(int a, int b);
void main(void){
    int a =0;
    int sum =0;
    while (a<10) {
        a= a+1;
        sum = adder (a,sum);
    }

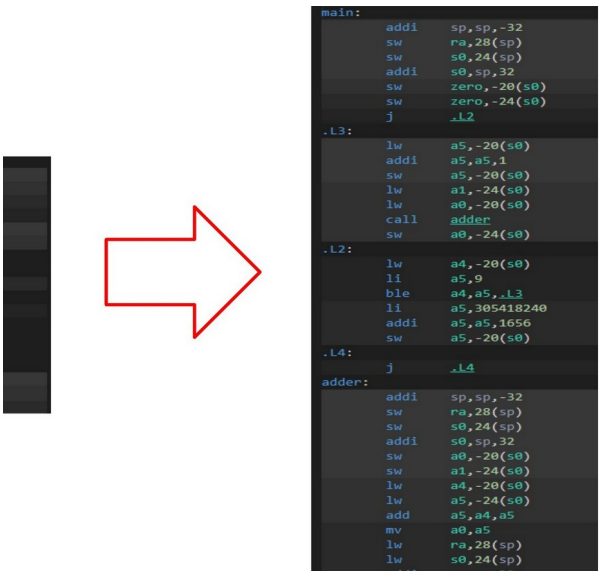
    a= 0x12345678;

    while(1); //halt

    return;
}

int adder(int a, int b) {
    return a+b;
}
```

[그림] 원본 C 코드 — while 루프 + adder 함수 호출 구조



```
main:
        addi    sp,sp,-32
        sw      ra,28(sp)
        sw      s0,24(sp)
        addi    s0,sp,32
        sw      zero,-20(s0)
        sw      zero,-24(s0)
        j       .L2
.L3:
        lw      a5,-20(s0)
        addi    a5,a5,1
        sw      a5,-20(s0)
        lw      a1,-24(s0)
        lw      a0,-20(s0)
        call    adder
        sw      a0,-24(s0)
.L2:
        lw      a4,-20(s0)
        li      a5,9
        ble     a4,a5,.L3
        li      a5,305418240
        addi    a5,a5,1656
        sw      a5,-20(s0)
.L4:
        j       .L4
adder:
        addi    sp,sp,-32
        sw      ra,28(sp)
        sw      s0,24(sp)
        addi    s0,sp,32
        sw      a0,-20(s0)
        sw      a1,-24(s0)
        lw      a4,-20(s0)
        lw      a5,-24(s0)
        add     a5,a4,a5
        mv      a0,a5
        lw      ra,28(sp)
        lw      s0,24(sp)
```

[그림] RISC-V 어셈블리 변환 결과 — main / .L3 / .L2 / .L4 / adder 섹션

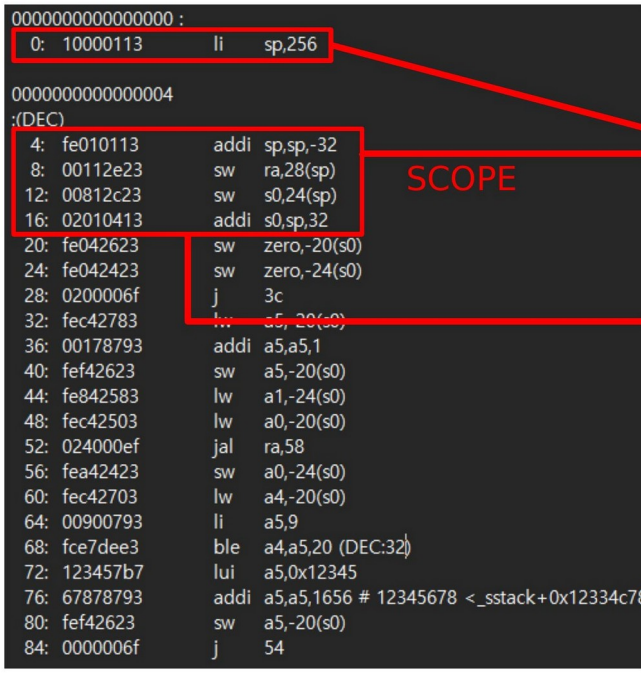
5.3 어셈블리 구조 및 기계어 코드

[코드] instruction_mem_sort.mem 기반 어셈블리 설명 (주요 구간 발췌)

```
; 1to10 adder - C 코드 → RISC-V 어셈블리 변환
; C 코드: int sum = 0; for(int i=1; i<=10; i++) sum += i;
; 레지스터 역할: a4(카운터 i), a5(누산기 sum), a0(최종 반환값)
;
; main 프레임 설정
0x00: 10000113  li   sp, 256        ; 스택 포인터 초기화
0x04: fe010113  addi sp, sp, -32    ; 스택 프레임 할당 (32 바이트)
```

```
0x08: 00112623  sw   ra, 28(sp)      ; 복귀 주소 저장
0x0C: 00812423  sw   s0, 24(sp)      ; s0 저장
0x10: 02010413  addi s0, sp, 32      ; 프레임 포인터 설정
0x14: fe042623  sw   zero, -20(s0)   ; sum = 0
0x18: fe042423  sw   zero, -24(s0)   ; i = 0 (초기화)
;
; 루프 호출
0x1C: 024000ef  jal  ra, 0x40        ; adder_func 호출
;
; adder_func 내부 루프 (a4=카운터, a5=누산기)
0x40: fe842783  lw   a5, -20(s0)     ; a5 = sum 로드
0x44: 00178793  addi a5, a5, 1       ; a5++
0x48: fef42623  sw   a5, -20(s0)     ; sum 저장
0x4C: fe842783  lw   a5, -20(s0)     ; a5 = sum
0x50: 00279793  slli a5, a5, 2       ; a5 <<= 2 (x4)
0x54: fdc42703  lw   a4, -36(s0)     ; a4 = base
0x58: 00f707b3  add  a5, a4, a5      ; a5 = base + sum*4
0x5C: 0007a703  lw   a4, 0(a5)       ; a4 = mem[a5]
; BLE 분기: a4 <= a5 일 때까지 반복
0x44: fce7dee3  ble  a4, a5, 0x20    ; if a4<=a5 goto loop
```

5.4 시뮬레이션 결과



[그림] 1to10 adder — main 프레임 어셈블리 코드



[그림] 파형 ① main 프레임 셋업 — sp=256, ra/s0 저장, zero 초기화

```
0000000000000000 :
   0:  10000113          li      sp,256

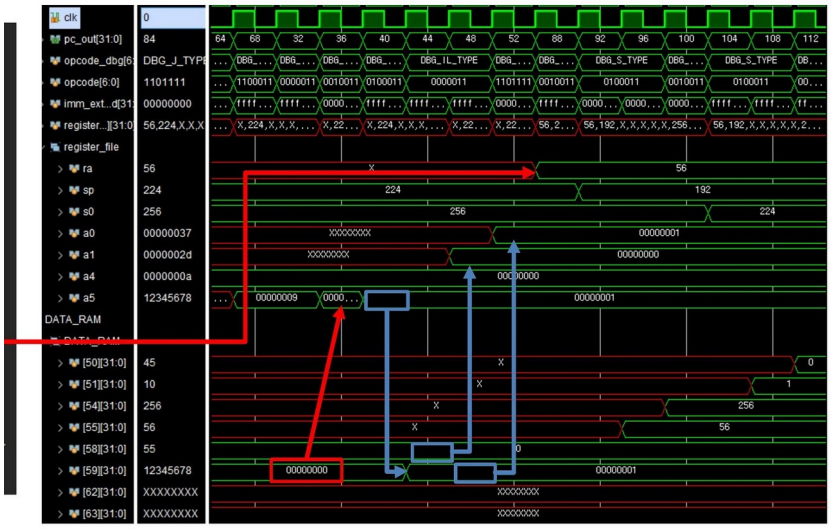
0000000000000004
:(DEC)
   4:  fe010113          addi    sp,sp,-32
   8:  00112e23          sw      ra,28(sp)
  12:  00812c23          sw      s0,24(sp)
  16:  02010413          addi    s0,sp,32
  20:  fe042623          sw      zero,-20(s0)
  24:  fe042423          sw      zero,-24(s0)
  28:  0200006f          j       3c
  32:  fec42783          lw      a5,-20(s0)
  36:  00178793          addi    a5,a5,1
  40:  fef42623          sw      a5,-20(s0)
  44:  fe842583          lw      a1,-24(s0)
  48:  fec42503          lw      a0,-20(s0)
  52:  024000ef          jal     ra,58
  56:  fea42423          sw      a0,-24(s0)
  60:  fec42703          lw      a4,-20(s0)
  64:  00900793          li      a5,9
  68:  fce7dee3          ble     a4,a5,20 (DEC:32)
  72:  123457b7          lui     a5,0x12345
  76:  67878793          addi    a5,a5,1656 # 12345678 <_sstack+0x12334c78
  80:  fef42623          sw      a5,-20(s0)
  84:  0000006f          j       54
```

(S0-20)/4 = 59

(S0-24)/4 = 58

Adder 함수 시작

[그림] 1to10 adder — JAL 호출 및 루프 어셈블리 코드



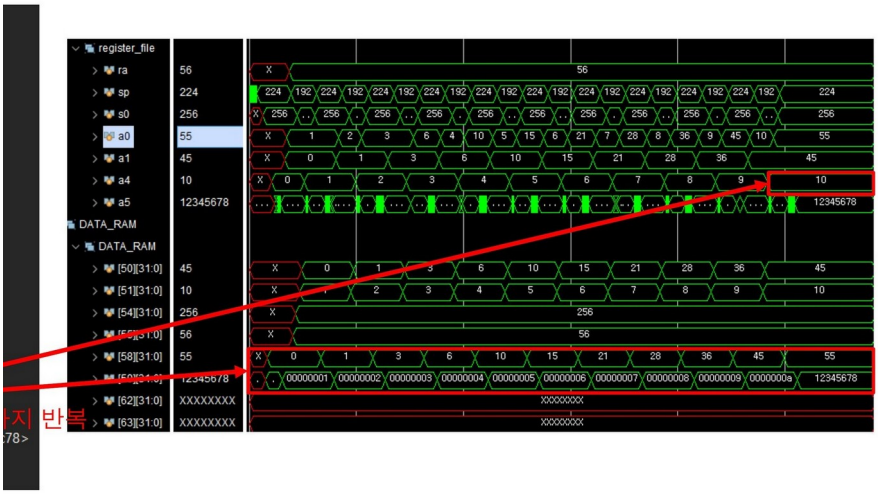
[그림] 파형 ② JAL → adder_func 호출, a4/a5 초기화

```
0000000000000000 :
   0:  10000113          li      sp,256

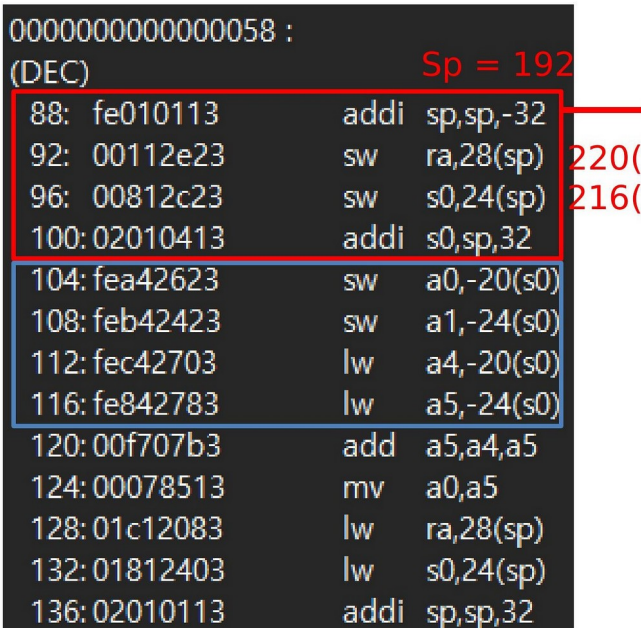
0000000000000004
:(DEC)
   4:  fe010113          addi    sp,sp,-32
   8:  00112e23          sw      ra,28(sp)
  12:  00812c23          sw      s0,24(sp)
  16:  02010413          addi    s0,sp,32
  20:  fe042623          sw      zero,-20(s0)
  24:  fe042423          sw      zero,-24(s0)
  28:  0200006f          j       3c
  32:  fec42783          lw      a5,-20(s0)
  36:  00178793          addi    a5,a5,1
  40:  fef42623          sw      a5,-20(s0)
  44:  fe842583          lw      a1,-24(s0)
  48:  fec42503          lw      a0,-20(s0)
  52:  024000ef          jal     ra,58
  56:  fea42423          sw      a0,-24(s0)
  60:  fec42703          lw      a4,-20(s0)
  64:  00900793          li      a5,9
  68:  fce7dee3          ble     a4,a5,20 (DEC:32)
  72:  123457b7          lui     a5,0x12345
  76:  67878793          addi    a5,a5,1656 # 12345678 <_sstack+0x1233
  80:  fef42623          sw      a5,-20(s0)
  84:  0000006f          j       54
```

a4=<a5 일때

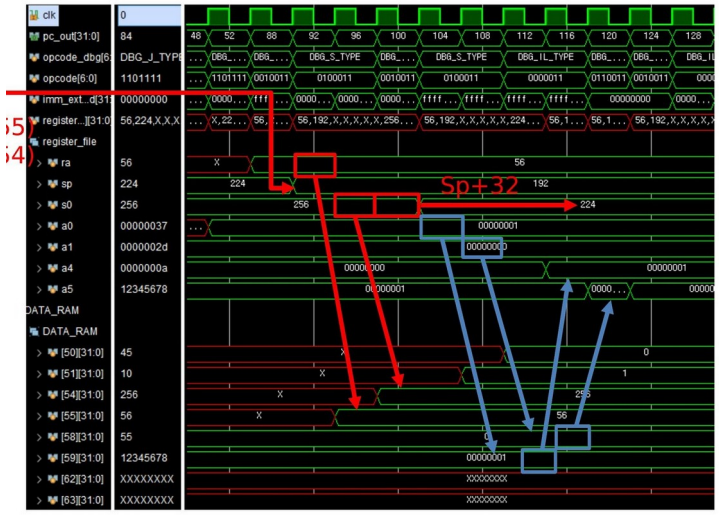
[그림] 1to10 adder — BLE 반복 루프 어셈블리 코드



[그림] 파형 ③ — a4(1~10 증가), a5(0→55 누산), DATA_RAM 변화



[그림] 1to10 adder — adder_func 복귀 어셈블리 코드



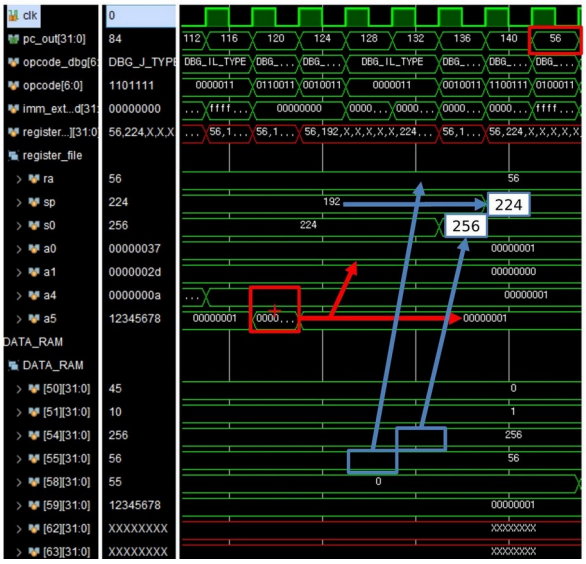
[그림] 파형 ④ adder_func 복귀 — Sp+32 복구, ret(JALR) 동작

```
0000000000000058 :
(DEC)
  88:  fe010113        addi  sp,sp,-32
  92:  00112e23        sw    ra,28(sp)
  96:  00812c23        sw    s0,24(sp)
 100: 02010413        addi  s0,sp,32
 104: fea42623        sw    a0,-20(s0)
 108: feb42423        sw    a1,-24(s0)
 112: fec42703        lw    a4,-20(s0)
 116: fe842783        lw    a5,-24(s0)
 120: 00f707b3        add   a5,a4,a5
 124: 00078513        mv    a0,a5
 128: 01c12083        lw    ra,28(sp)
 132: 01812403        lw    s0,24(sp)
 136: 02010113        addi  sp,sp,32
```

220
216

[그림] 1to10 adder — 최종 결과 반환 어셈블리 코드

55)
54)



[그림] 최종 결과 파형 — a0=55 반환값, DATA_RAM[58]=55, sp/s0 복구

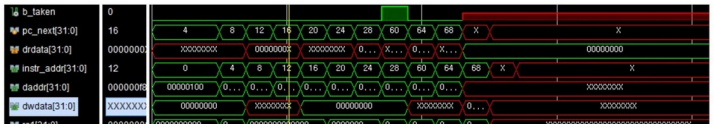
결과 분석: a0(최종 반환값)에 10 사이클에 걸쳐 1+2+3+...+10=55 가 올바르게 저장됨을 파형으로 확인. a4(카운터)가 1→2→...→10 으로 증가하고, a5(누산기)가 1→3→6→10→15→21→28→36→45→55 로 누적되는 과정이 DATA_RAM 및 레지스터 파형에서 명확히 관찰됨.

6. 결과 분석 및 트러블슈팅 (Analysis & Trouble Shooting)

6.1 b_taken X 값 전파 문제

B-Type 명령어 검증 과정에서 시뮬레이션 초기에 rs1 또는 rs2 가 초기화되지 않은 상태(X 값)일 때 b_taken 이 X(부정값)로 출력되어 pc_next 가 불확정 상태가 되는 문제가 발견되었다.

구분	내용
발생 상황	B-Type 명령어 실행 시 rs1 또는 rs2 가 X 상태이면 비교 회로 출력 b_taken 도 X 로 출력됨. PC SRC MUX 의 sel 이 X 가 되어 pc_next 가 X(부정값) 상태로 전파되어 이후 모든 동작이 불확정
근본 원인	기존 case-expression 방식(b_taken = (rs1==rs2)?1:0)은 피연산자에 X 값이 포함될 경우 비교 결과가 X 로 전파됨. 삼항 연산자의 조건이 X 이면 SystemVerilog 에서 출력도 X 가 되는 시뮬레이션 시맨틱 문제
해결 방법	case 내부를 if-else 명시적 분기 구조로 변경. 모든 분기에서 b_taken = 1'b1 또는 b_taken = 1'b0 을 명시적으로 할당하여 X 입력이 있어도 확정값(0 또는 1)을 출력하도록 수정
검증 결과	수정 후 B-Type 전 명령어(BEQ~BGEU) 재검증. b_taken 이 0 또는 1 의 확정값만 출력되어 PC 분기 동작 정상 확인. 1to10 adder 통합 시뮬레이션도 정상 완료



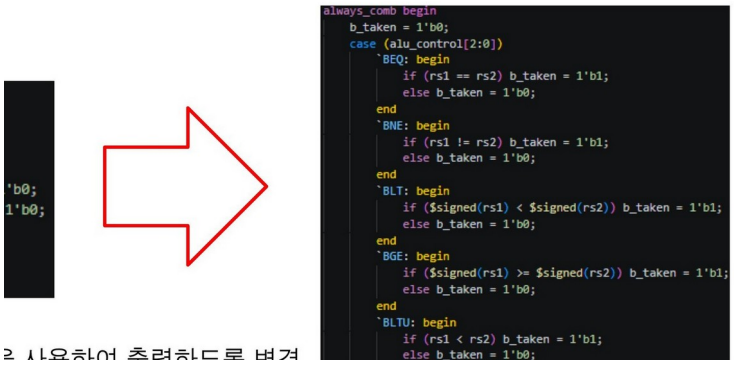
[그림] Trouble Shooting 수정 전 파형 — b_taken=X, pc_next=X 불확정 전파

[코드] rv32i_datapath.sv — b_taken 수정 전 (case-expression, X 값 전파 발생)

```
// [수정 전 - X값 전파 문제 발생]
always_comb begin
    b_taken = 1'b0;
    case (alu_control[2:0])
        `BEQ:  b_taken = (rs1 == rs2) ? 1'b1 : 1'b0;
        `BNE:  b_taken = (rs1 != rs2) ? 1'b1 : 1'b0;
        `BLT:  b_taken = ($signed(rs1) < $signed(rs2)) ? 1'b1 : 1'b0;
        `BGE:  b_taken = ($signed(rs1) >= $signed(rs2)) ? 1'b1 : 1'b0;
        `BLTU: b_taken = (rs1 < rs2) ? 1'b1 : 1'b0;
        `BGEU: b_taken = (rs1 >= rs2) ? 1'b1 : 1'b0;
    endcase
end
```

```
always_comb begin
    b_taken = 1'b0;
    case (alu_control[2:0])
        `BEQ:  b_taken = (rs1 == rs2) ? 1'b1 : 1'b0;
        `BNE:  b_taken = (rs1 != rs2) ? 1'b1 : 1'b0;
        `BLT:  b_taken = ($signed(rs1) < $signed(rs2)) ? 1'b1 :
        `BGE:  b_taken = ($signed(rs1) >= $signed(rs2)) ? 1'b1 :
        `BLTU: b_taken = (rs1 < rs2) ? 1'b1 : 1'b0;
        `BGEU: b_taken = (rs1 >= rs2) ? 1'b1 : 1'b0;
    endcase
end
```

[그림] 수정 전 코드 — case-expression 방식 (X 값 전파 발생)



[그림] 수정 후 코드 — if-else 명시적 분기 (X 값 전파 방지)

[코드] rv32i_datapath.sv — b_taken 수정 후 (if-else, X 값 전파 방지)

```
always_comb begin
    b_taken = 1'b0;
    case (alu_control[2:0])
        `BEQ: begin
            if (rs1 == rs2) b_taken = 1'b1;
            else b_taken = 1'b0;
        end
        `BNE: begin
            if (rs1 != rs2) b_taken = 1'b1;
            else b_taken = 1'b0;
        end
        `BLT: begin
            if ($signed(rs1) < $signed(rs2)) b_taken = 1'b1;
            else b_taken = 1'b0;
        end
        `BGE: begin
            if ($signed(rs1) >= $signed(rs2)) b_taken = 1'b1;
            else b_taken = 1'b0;
        end
        `BLTU: begin
            if (rs1 < rs2) b_taken = 1'b1;
            else b_taken = 1'b0;
        end
        `BGEU: begin
            if (rs1 >= rs2) b_taken = 1'b1;
            else b_taken = 1'b0;
        end
    endcase
end
```



[그림] 수정 전 — b_taken=X (불확정)

[그림] 수정 후 — b_taken=0/1 확정값, pc_next 정상 동작

6.2 J-Type 즉시값 비트 재조합 오류

구분	내용
발생 상황	J-Type(JAL) 명령어 실행 시 점프 주소가 예상값과 다르게 출력되는 오류 발생
원인 분석	RISC-V J-Type 즉시값은 instr[31](imm[20]), instr[19:12](imm[19:12]), instr[20](imm[11]), instr[30:21](imm[10:1]) 순으로 분산 인코딩됨. 초기 구현에서 비트 순서 오기재
해결 방법	RISC-V 스펙 재확인 후 imm_extend 모듈의 J-Type 비트 연결 순서 수정: {12{instr[31]}}, instr[19:12], instr[20], instr[30:21], 1'b0}

6.3 data_mem drdata 중복 선언 오류

구분	내용
발생 상황	data_mem 모듈의 drdata 출력이 always_ff 와 always_comb 두 블록에서 동시에 구동되어 시뮬레이션 오류 발생
해결 방법	always_ff 블록의 drdata 할당을 제거하고 always_comb 단일 블록으로 일원화. 메모리 읽기는 비동기로 충분하며, 순차 논리가 필요하지 않음을 확인

7. 결론 및 고찰

7.1 설계 결과 요약

본 프로젝트에서는 RISC-V RV32I 명령어 셋의 9 가지 타입(R/I/S/L/B/UL/UA/J/JL) 36 개 명령어를 완전히 지원하는 Single Cycle CPU 를 SystemVerilog 로 구현하고, 타입별 시뮬레이션 및 1to10 adder 통합 검증을 통해 정상 동작을 확인하였다.

항목	결과
지원 명령어 타입	R / I(산술) / I(Load) / S / B / UL(LUI) / UA(AUIPC) / J(JAL) / JL(JALR) — 9 종
검증 명령어 수	R(10) + I(8) + S(3) + L(5) + B(6) + U(2) + J(1) + JL(1) = 총 36 개
통합 프로그램 검증	1to10 adder — C 코드 for 루프를 RISC-V 어셈블리로 변환. 최종 a0=55 정상 출력
트러블슈팅	b_taken X 값 전파 문제 (case→if-else 수정), J-Type 비트 재조합 오류, drdata 중복 구동 오류 해결
시뮬레이션 환경	10ns 클럭(50MHz), 2 사이클 rst, 20 사이클+ 자유 실행. $readmemh 기반 명령어 로드

7.2 고찰

1. RV32I 를 구현하면서 BEQ 같은 분기 명령이 단순 비교가 아니라 PC 변경과 연결된다는 점을 이해하게 되었고, 데이터 흐름 전체를 고려해야 정상 동작한다는 것을 알게 되었다.

2. C 언어로 작성된 코드를 RV32I 구조로 변환하는 과정에서 한 줄의 코드도 여러 하드웨어 동작 (레지스터 읽기, ALU 연산, 메모리 접근) 으로 나뉜다는 점을 체감했다.

3. 전체 설계에서 작은 문법 오류보다도 구조 (데이터 흐름, 신호 연결) 를 제대로 잡는 것이 더 중요하다는 것을 느꼈다.

[그림] 고찰 — 프로젝트를 통해 얻은 3 가지 핵심 인사이트

- BEQ 같은 분기 명령이 단순한 값 비교가 아니라 PC 변경 경로 전체와 연결된다는 점을 설계하면서 직접 체감하였다. b_taken 신호 하나가 PC SRC MUX 를 통해 실행 흐름 전체를 바꾸는 구조를 이해함으로써, 분기 예측 및 파이프라인 해저드 문제가 왜 발생하는지에 대한 기초적 이해를 얻었다.
- C 언어 한 줄(예: sum += i)이 레지스터 읽기 → ALU 덧셈 → 결과 저장 → 메모리 접근으로 여러 하드웨어 동작으로 분해된다는 점을 1to10 adder 구현을 통해 직접 확인하였다. 이는 컴파일러가 수행하는 최적화의 필요성과 한계를 하드웨어 레벨에서 이해하는 데 도움이 되었다.
- 전체 설계에서 작은 문법 오류보다 데이터 흐름과 신호 연결 구조를 정확히 설계하는 것이 더 중요하다는 점을 절감하였다. b_taken X 값 전파 문제와 J-Type 즉시값 비트 오기재 문제 모두 구조적 이해 부족에서 비롯된 것이었으며, RTL 설계는 소프트웨어 코딩과 달리 신호의 타이밍과 전파 경로를 입체적으로 사고해야 함을 배웠다.
- Single Cycle 구조의 가장 큰 제약은 가장 느린 명령어(예: Load — 메모리 접근 포함)에 맞춰 클럭 주기를 설정해야 한다는 것이다. 이로 인해 빠른 명령어들이 불필요하게 긴 클럭 주기를 기다리게 되며, 이 문제를 해결하기 위해 파이프라인 방식이 도입된다는 점을 이론이 아닌 설계를 통해 이해하게 되었다.

7.3 향후 개선 방향

- 파이프라인(Pipeline) 구조로 확장 — IF/ID/EX/MEM/WB 5 단계 분리, 데이터 해저드 및 제어 해저드 처리

- 캐시(Cache) 메모리 추가 — 현재 단일 사이클 메모리 접근을 다단계 캐시 계층으로 개선
- RV32M 확장 명령어 지원 — MUL, DIV, REM 등 정수 곱셈·나눗셈 명령어 추가
- FPGA 구현 — 시뮬레이션 환경에서 검증된 설계를 Xilinx/Altera FPGA 에 합성 및 실제 동작 확인

참고 문헌

[0] RISC-V International, "The RISC-V Instruction Set Manual, Volume I: Unprivileged ISA", Version 20191213, 2019.

[1] Patterson, D. A. & Hennessy, J. L., "Computer Organization and Design RISC-V Edition: The Hardware Software Interface", Morgan Kaufmann, 2017.

[2] IEEE, "IEEE Standard for SystemVerilog — Unified Hardware Design, Specification, and Verification Language", IEEE Std 1800-2017, 2017.

[3] Waterman, A. & Asanovic, K., "The RISC-V Instruction Set Manual, Volume II: Privileged Architecture", Version 20211203, RISC-V Foundation, 2021.